

Laporan Milestone 2 Syntax Analysis

IF2224 TEORI BAHASA FORMAL DAN OTOMATA



Disusun oleh:

BST – brawlstars

Kevin Wirya Valerian 13524019

Ray Owen Martin 13524033

Emilio Justin 13524043

Philipp Hamara 13524101

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

Daftar Isi.....	2
Daftar Gambar.....	3
I. Teori Singkat.....	4
A. Parser.....	4
B. Parse Tree.....	4
C. Algoritma Recursive Descent.....	6
II. Perancangan dan Implementasi.....	8
A. Revisi Milestone 1.....	8
B. Rancangan Parse Tree.....	8
C. Implementasi Program.....	9
III. Pengujian.....	11
A. Pengujian 1.....	11
B. Pengujian 2.....	12
C. Pengujian 3.....	17
D. Pengujian 4.....	23
E. Pengujian 5.....	26
F. Pengujian 6.....	44
G. Pengujian 7.....	45
H. Pengujian 8.....	45
I. Pengujian 9.....	47
J. Pengujian 10.....	48
IV. Kesimpulan dan Saran.....	50
A. Kesimpulan.....	50
B. Saran.....	50
Referensi.....	52
Lampiran.....	53

Daftar Gambar

Gambar 1.1 Gambar Parse Tree 1..... 5

I. Teori Singkat

A. Parser

Secara umum, parser adalah suatu program yang melakukan interpretasi stream bit fisik suatu pesan mendatang dan menciptakan suatu representasi logis internal dari pesan tersebut dalam suatu struktur pohon. Parser juga menghasilkan sebuah stream bit selain daripada hasil representasi pohon tersebut. Definisi ini umumnya digunakan dan dijelaskan untuk parser di bidang web development.

Dalam bidang teori automata, parser memiliki definisi yang lebih spesifik, yakni program yang melakukan analisa sejumlah token untuk menentukan struktur sintaks dari suatu program. Parser tetap harus memperhatikan aturan tata bahasa dari bahasa pemrograman yang sedang dikompilasi. Token yang diterima oleh parser akan diproses menjadi bentuk pohon (parse tree atau abstract syntax tree (AST)).

Dengan kata lain, peran parser dapat disimpulkan menjadi:

- Memeriksa kepatuhan struktur kode program terhadap aturan tata bahasa dari bahasa pemrograman yang dikompilasi
- Menuntun dan mendukung analisa context-sensitive, yang membantu memverifikasi aturan berbasis makna seperti pengecekan struktur data
- Membentuk struktur parse tree sebagai representasi langsung
- Menghasilkan pesan error yang bermakna, seperti sintaks error
- Memperbaiki error minimal jika ditemukan pada kode program

Tentunya peran-peran ini tidak dapat terpisah dari peran lexer yang menerima kode program dan diolah menjadi token-token. Token-token ini menjadi input bagi parser, sehingga krusial bagi lexer untuk melakukan tokenizing yang tepat (bahkan sempurna) agar parser dapat menjalankan proses parsing dengan baik pula.

B. Parse Tree

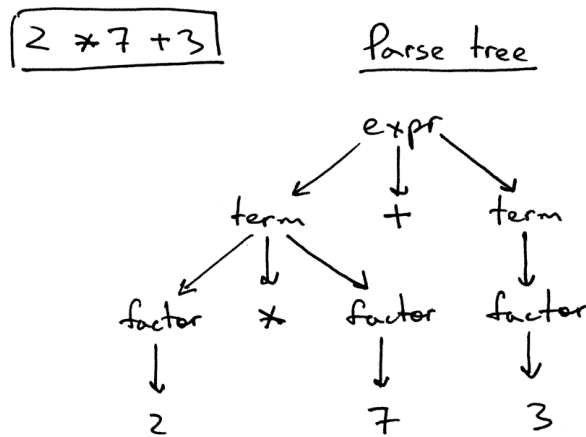
Parse tree, disebut juga syntax tree, adalah representasi hasil derivasi suatu string berdasarkan aturan tata bahasa pada bahasa pemrograman terkait. Seperti yang dijelaskan pada bagian sebelumnya, parse tree adalah output dari parser. Parse tree menunjukkan bagaimana suatu start symbol ditransformasi menjadi string

terminal dengan menggunakan aturan produksi (aturan tata bahasa) secara berurutan.

Parse tree memiliki beberapa kegunaan berikut:

- Membantu pembuatan syntax analysis dengan merefleksikan sintaks bahasa input
- Menggunakan in-memory representation hasil mengubah struktur teks agar lebih mudah diproses
- Memproses informasi berulang kali tanpa harus re-parsing

Parse tree juga digunakan untuk melakukan visualisasi struktur kode program untuk memastikan struktur tersebut sudah sesuai dengan aturan bahasa pemrograman. Parse tree juga penting untuk deteksi error dan optimisasi, oleh karena itu parse tree penting dalam membuat suatu compiler lebih efisien.



Gambar 1.1 Gambar Parse Tree 1
(Sumber: <https://ruslanspivak.com/lsbasi-part7>)



C. Algoritma Recursive Descent

Algoritma ini memiliki keunikan, yaitu penurunan semua variabel dari awal hingga akhir (terminal) secara rekursi. Suatu tipe dari recursive descent parser, predictive parser, tidak memerlukan backtracking, berbeda dengan recursive descent murni yang tetap menggunakan backtracking. Ini menyebabkan algoritma recursive decent sederhana dan dapat dibangun dari recognizers hanya dengan bantuan tambahan beberapa pekerjaan spesifik. Meskipun demikian, algoritma sederhana ini tetap memiliki kelemahan, seperti tidak secepat algoritma lain dan sulit membuat error message yang baik. Algoritma ini juga kesulitan menghadapi grammar left-recursive.

Algoritma ini terdiri dari beberapa tahapan:

- Inisialisasi dan penentuan lookahead, yaitu token yang sedang dianalisis saat ini, direpresentasikan oleh sebuah pointer.
- Evaluasi aturan utama (root start) dengan memanggil fungsi yang merepresentasikan grammar paling tinggi
- Pencocokan token terminal, jika match, maka parser akan melakukan consume dan memproses nilainya, lalu memajukan pointer lookahead, jika tidak maka ini indikasi syntax error
- Pemanggilan fungsi rekursif untuk simbol non-terminal. Bagian ini merepresentasikan aspek rekursif. Pada bagian ini, parser akan mendelegasikan tugas tersebut dengan memanggil fungsi spesifik yang menangani aturan tersebut.
- Pembentukan simpul pohon ketika hasil validasi suatu fungsi sudah ada, yang menandakan fungsi tersebut sudah menerima output dari fungsi rekursif di bawahnya
- Proses ini akan berlangsung terus-menerus hingga mencapai akhir dari antrean list of tokens.

II. Perancangan dan Implementasi

A. Revisi Milestone 1

Dalam pengerjaan parser dan Milestone 2, diperlukan revisi dalam beberapa aspek-aspek pada source code Lexer di Milestone 1. Terutama, adalah perubahan source code sehingga menyimpan vector/list of tokens dari lexer dan penggunaan struct Tokens. Pada milestone 1, hasil hanya berupa output tokens dalam bentuk .txt tanpa disimpan dalam runtime program. Untuk integrasi dengan parser, dilakukan revisi untuk menggunakan dan menyimpan tokens pada runtime untuk mengonstruksikan pipeline dari input user menjadi hasil parse tree. Terdapat juga penambahan token 'Unknown' untuk memindahkan tanggung jawab error-handling ke dalam tangan parser.

B. Rancangan Parse Tree

Secara formal, parse tree yang dirancang dalam program ini digunakan untuk merepresentasikan struktur sintaks dari bahasa pemrograman Arion berdasarkan aturan grammar yang telah ditentukan. Parse tree dibangun menggunakan struktur node bertipe ParseNode, di mana setiap node merepresentasikan simbol dalam grammar, baik terminal maupun non-terminal.

Setiap node memiliki label dan sekumpulan child node yang membentuk hierarki pohon. Node dengan label seperti `<program>`, `<expression>`, dan `<statement>` merupakan non-terminal, sedangkan token seperti `ident`, `+`, atau `:=` direpresentasikan sebagai leaf node.

Struktur parse tree mengikuti aturan grammar secara top-down. Node akar (root) dari parse tree adalah `<program>`, yang memiliki empat komponen utama, yaitu `<program-header>`, `<declaration-part>`, `<compound-statement>`, dan simbol terminal period. Dengan demikian, parse tree secara langsung mencerminkan produksi grammar:

`program` $\rightarrow$ `program-header declaration-part compound-statement .`

Untuk ekspresi, parse tree dirancang dengan memperhatikan prioritas operator. Struktur hierarki dibagi menjadi beberapa level, yaitu `<expression>`, `<simple-expression>`, `<term>`, dan `<factor>`. Pembagian ini memastikan bahwa operator dengan prioritas lebih tinggi (seperti perkalian) diproses lebih dahulu dibandingkan operator dengan prioritas lebih rendah (seperti penjumlahan).

Sebagai contoh, ekspresi $a + b * c$ akan menghasilkan parse tree yang menempatkan operasi $b * c$ lebih dahulu dibandingkan $a + \dots$, sehingga urutan evaluasi tetap benar.

Selain itu, parse tree juga mendukung berbagai struktur kontrol seperti percabangan (if), perulangan (while, for, repeat), serta pemanggilan prosedur dan fungsi. Setiap konstruksi direpresentasikan sebagai subtree tersendiri dengan node khusus, seperti `<if-statement>` atau `<while-statement>`.

C. Implementasi Program

1. Implementasi Parser

Implementasi parser dalam program ini menggunakan metode recursive descent parsing, di mana setiap aturan grammar direpresentasikan sebagai sebuah fungsi dalam kode. Fungsi-fungsi seperti `parseProgram`, `parseStatement`, dan `parseExpression` secara langsung mencerminkan produksi grammar yang ada.

Parser bekerja dengan membaca token hasil dari lexical analyzer secara berurutan menggunakan pointer posisi (`pos`). Token saat ini dapat diakses melalui fungsi `cur()`, sedangkan token berikutnya dapat dilihat menggunakan fungsi `peek()` untuk keperluan lookahead.

Struktur kontrol utama parser menggunakan pendekatan top-down, dimulai dari fungsi `parseProgram()` sebagai entry point. Fungsi ini kemudian memanggil fungsi lain sesuai dengan struktur grammar hingga seluruh token selesai diproses.

2. Pengenalan Struktur Sintaks

Setiap bagian dari grammar diimplementasikan sebagai fungsi parsing yang menghasilkan node parse tree. Sebagai contoh:

- `parseAssignmentStatement` menangani assignment seperti $x := y + 1$

- `parseIfStatement` menangani percabangan if-then-else
- `parseWhileStatement` menangani perulangan while.

Untuk membedakan antara assignment dan pemanggilan fungsi, parser menggunakan teknik lookahead. Jika sebuah ident diikuti oleh token `:=`, maka akan diproses sebagai assignment. Jika tidak, maka akan dianggap sebagai pemanggilan prosedur atau fungsi.

3. Penanganan Ekspresi

Ekspresi ditangani secara bertingkat untuk menjaga prioritas operator. Fungsi `parseExpression` akan memanggil `parseSimpleExpression`, yang kemudian memanggil `parseTerm`, dan akhirnya `parseFactor`.

Operator relasional seperti `=`, `<`, dan `>` diproses pada level `<expression>`, sedangkan operator penjumlahan dan pengurangan diproses pada level `<simple-expression>`, dan operator perkalian serta pembagian diproses pada level `<term>`.

Pendekatan ini memastikan bahwa struktur parse tree yang dihasilkan sesuai dengan aturan precedence dan associativity dalam bahasa pemrograman.

4. Error Handling

Error handling dalam parser dilakukan menggunakan fungsi `addError`, yang akan mencatat pesan kesalahan ketika ditemukan token yang tidak sesuai dengan aturan grammar.

Selain itu, fungsi `expect` digunakan untuk memastikan bahwa token yang dibaca sesuai dengan yang diharapkan. Jika tidak, parser akan mencatat error dan tetap melanjutkan parsing dengan membuat node placeholder seperti `[missing:token]`.

Pendekatan ini memungkinkan parser untuk tetap menghasilkan parse tree meskipun terdapat kesalahan sintaks, sehingga memudahkan proses debugging.

5. Visualisasi Parse Tree

Parse tree yang dihasilkan dapat ditampilkan menggunakan fungsi `printTree`, yang mencetak struktur pohon secara hierarkis menggunakan simbol seperti `|` dan `└`. Visualisasi ini membantu dalam memahami struktur sintaks program secara lebih intuitif.

III. Pengujian

A. Pengujian 1

Input

```

program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.

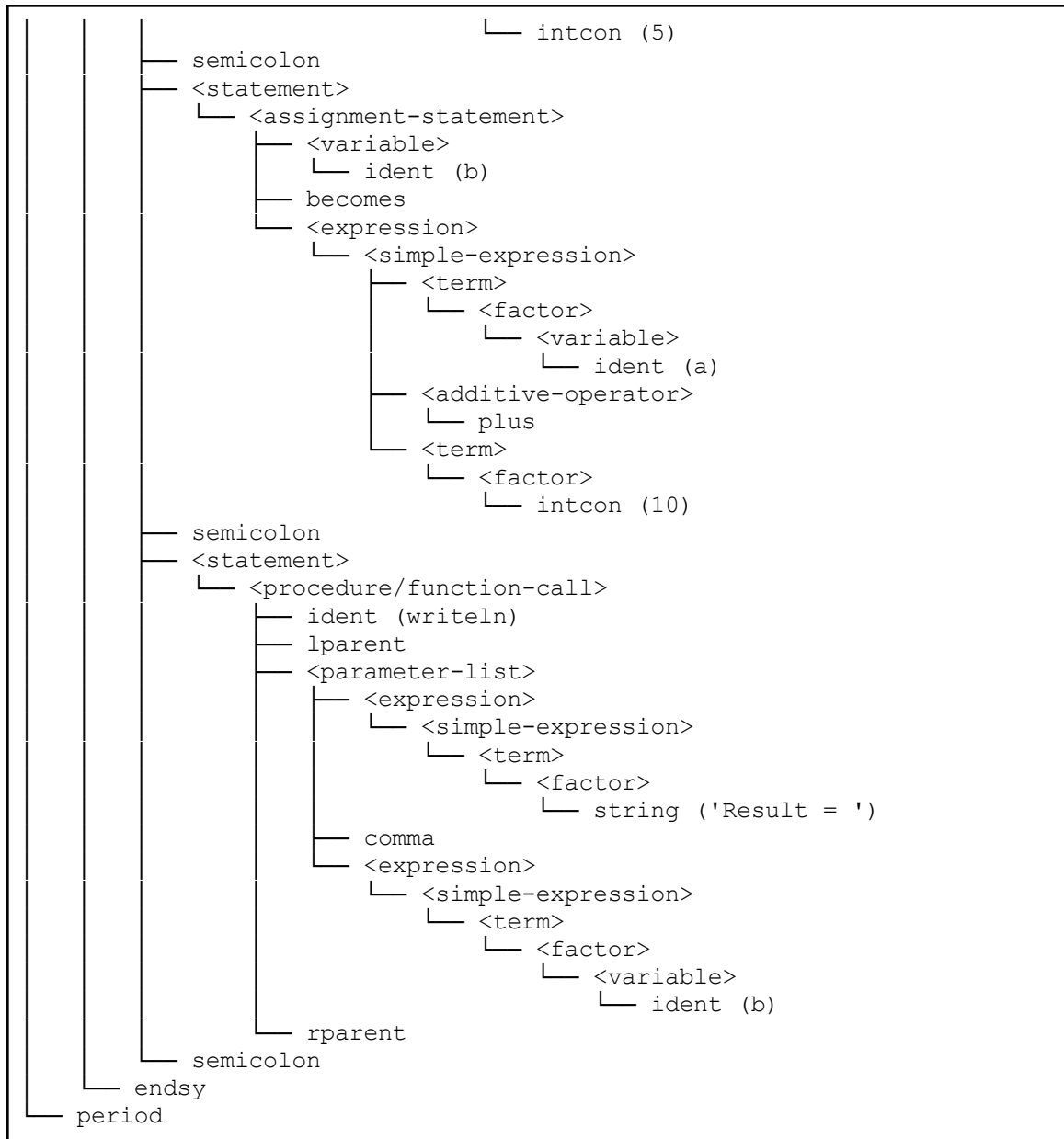
```

Output

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Hello)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (a)
│       │   ├── comma
│       │   └── ident (b)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon
└── <compound-statement>
    ├── beginsy
    └── <statement-list>
        └── <statement>
            └── <assignment-statement>
                ├── <variable>
                │   └── ident (a)
                ├── becomes
                └── <expression>
                    ├── <simple-expression>
                    │   ├── <term>
                    │   │   └── <factor>

```



B. Pengujian 2

Input

```

program Subprograms;
var

```

```

    i, total: integer;

procedure AddToTotal(x: integer; y: integer);
begin
    total := total + x + y;
end;

function NextVal(v: integer): integer;
begin
    NextVal := v + 1;
end;

begin
    i := 1;
    total := 0;
    AddToTotal(i, NextVal(i));
    if total > 0 then
        total := total + 1
    else
        total := total - 1;
    end.

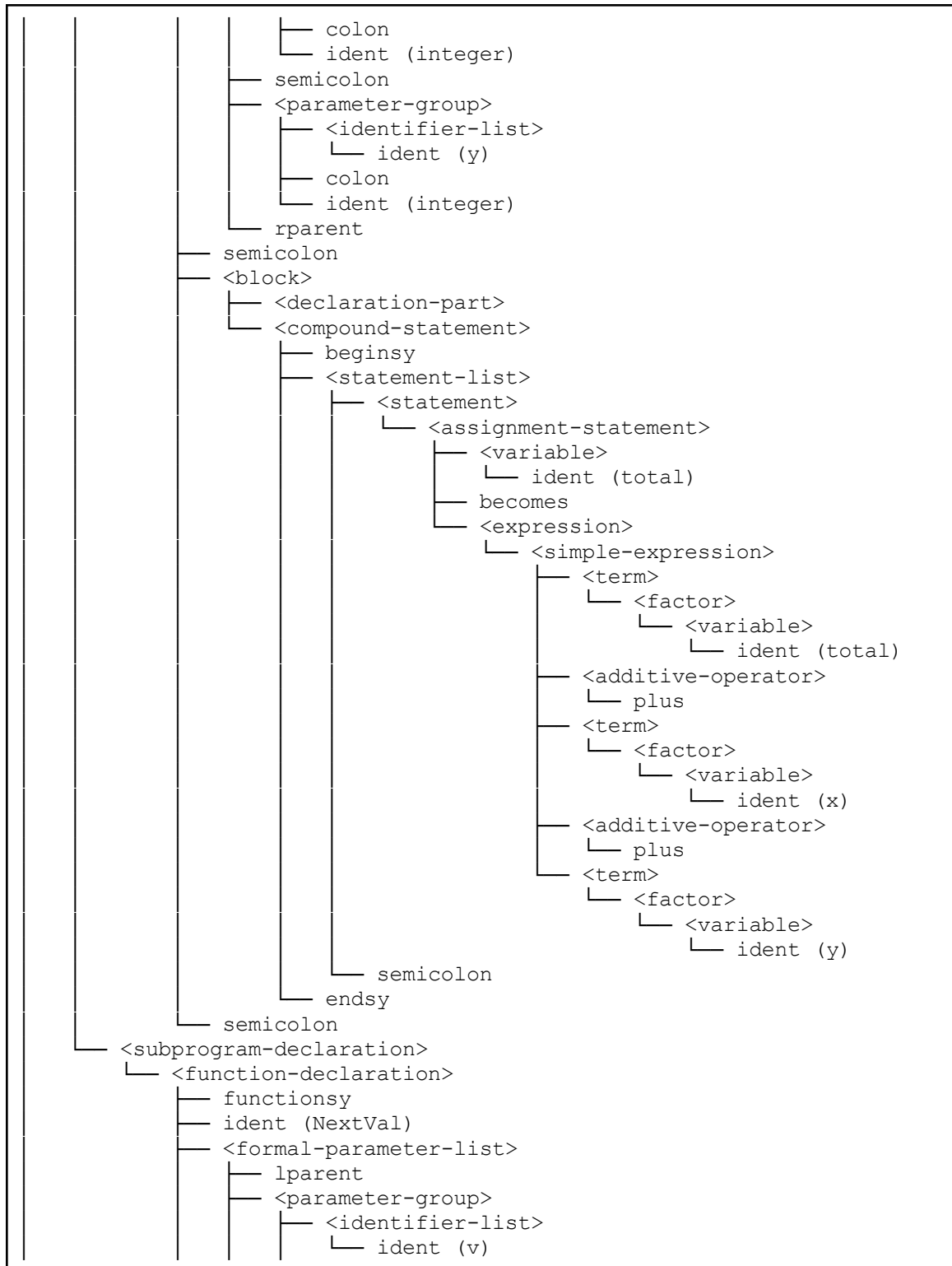
```

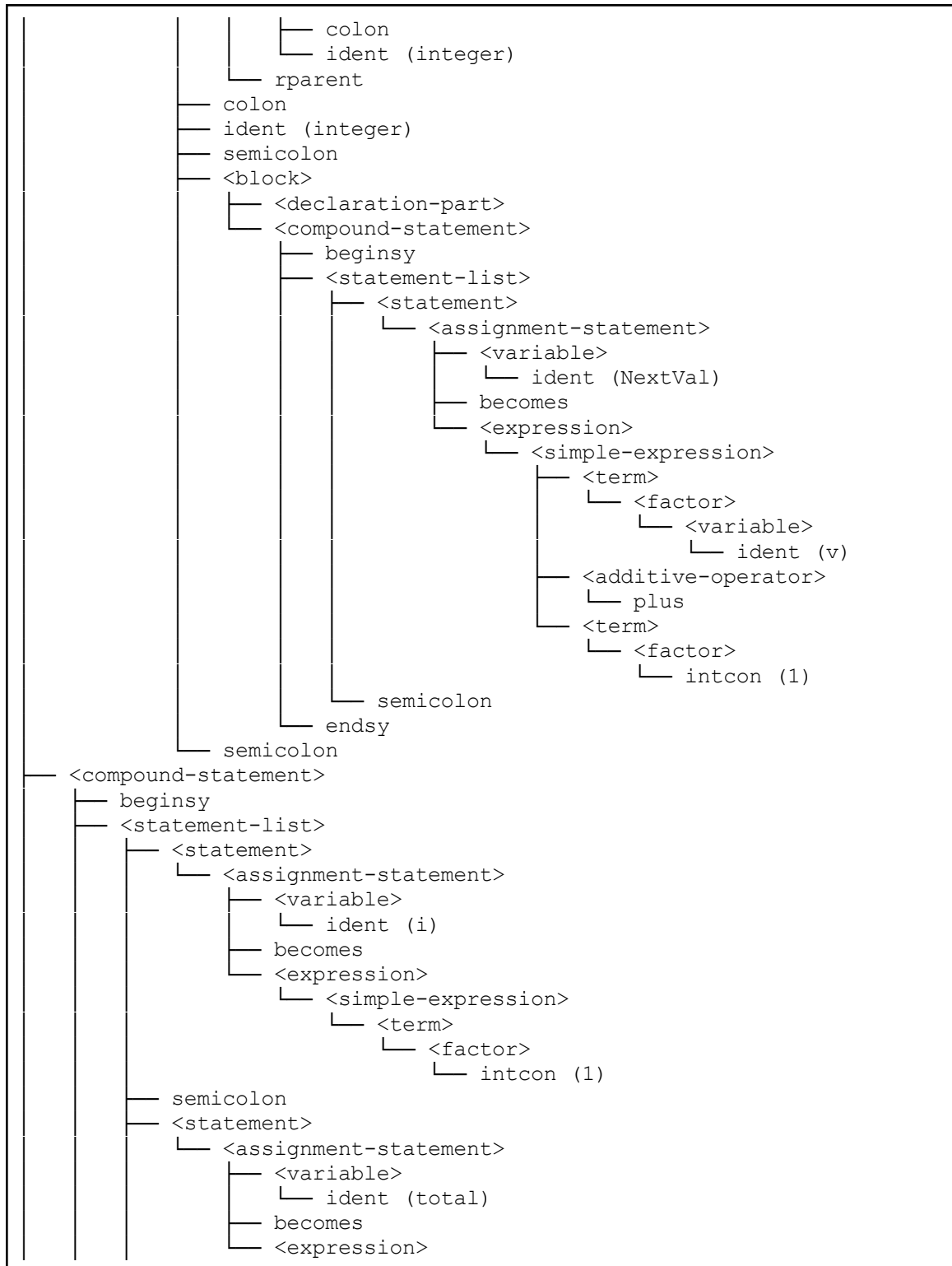
Output

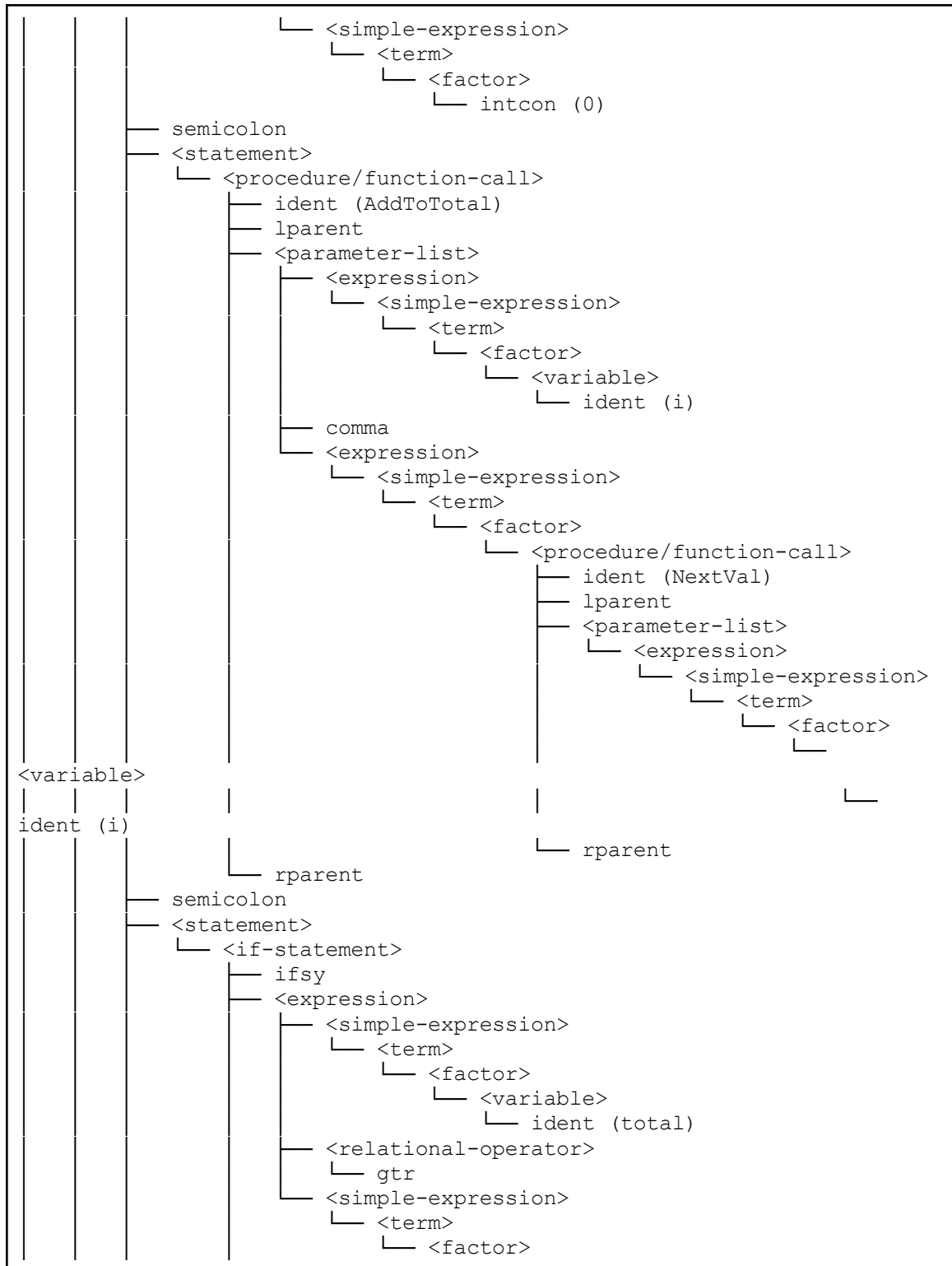
```

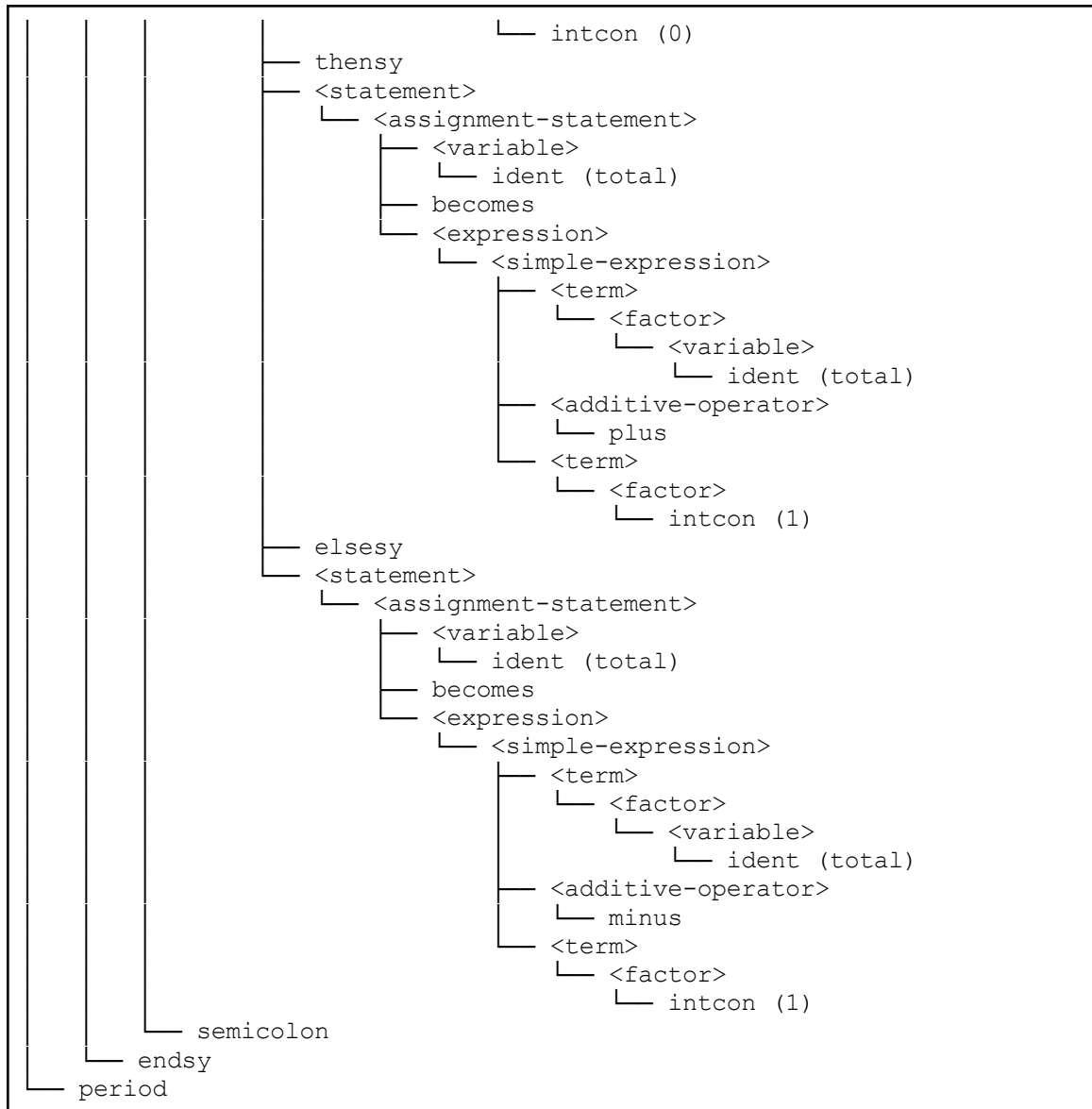
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Subprograms)
│   └── semicolon
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── varsy
│   │   ├── <identifier-list>
│   │   │   ├── ident (i)
│   │   │   ├── comma
│   │   │   └── ident (total)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (integer)
│   │   └── semicolon
│   └── <subprogram-declaration>
│       ├── <procedure-declaration>
│       │   ├── proceduresy
│       │   ├── ident (AddToTotal)
│       │   ├── <formal-parameter-list>
│       │   │   ├── lparent
│       │   │   ├── <parameter-group>
│       │   │   │   ├── <identifier-list>
│       │   │   │   └── ident (x)

```









C. Pengujian 3

Input

```

program ControlFlow;
var
  i, x: integer;

```

```

begin
  x := 0;
  for i := 1 to 3 do
    x := x + i;

  while x > 0 do
    x := x - 1;

  repeat
    x := x + 2;
  until x > 1;

  if x > 1 then
    x := 5
  else
    x := 6;

  case x of
    1: x := 10;
    2, 3: x := 20;
  end;
end.

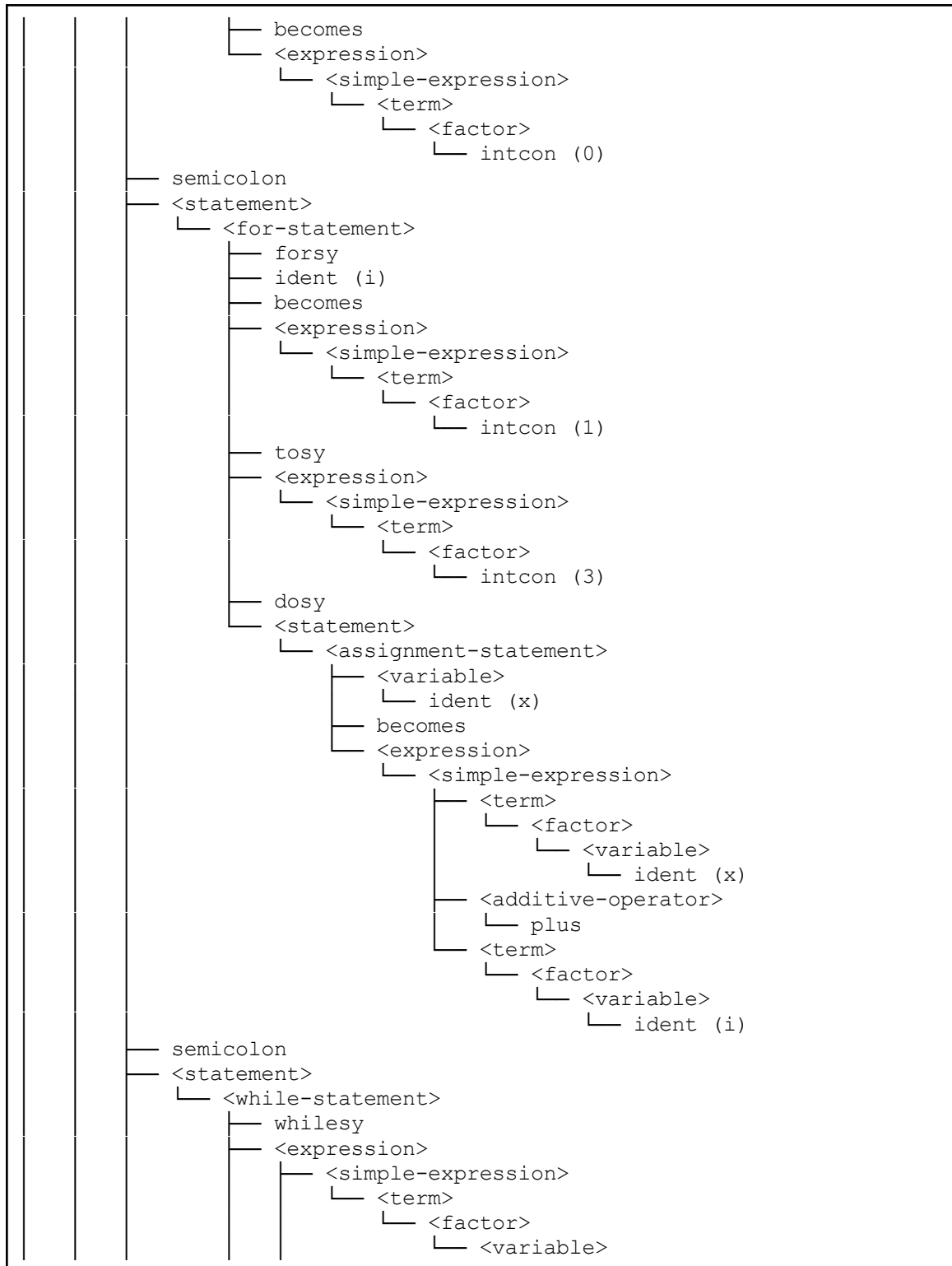
```

Output

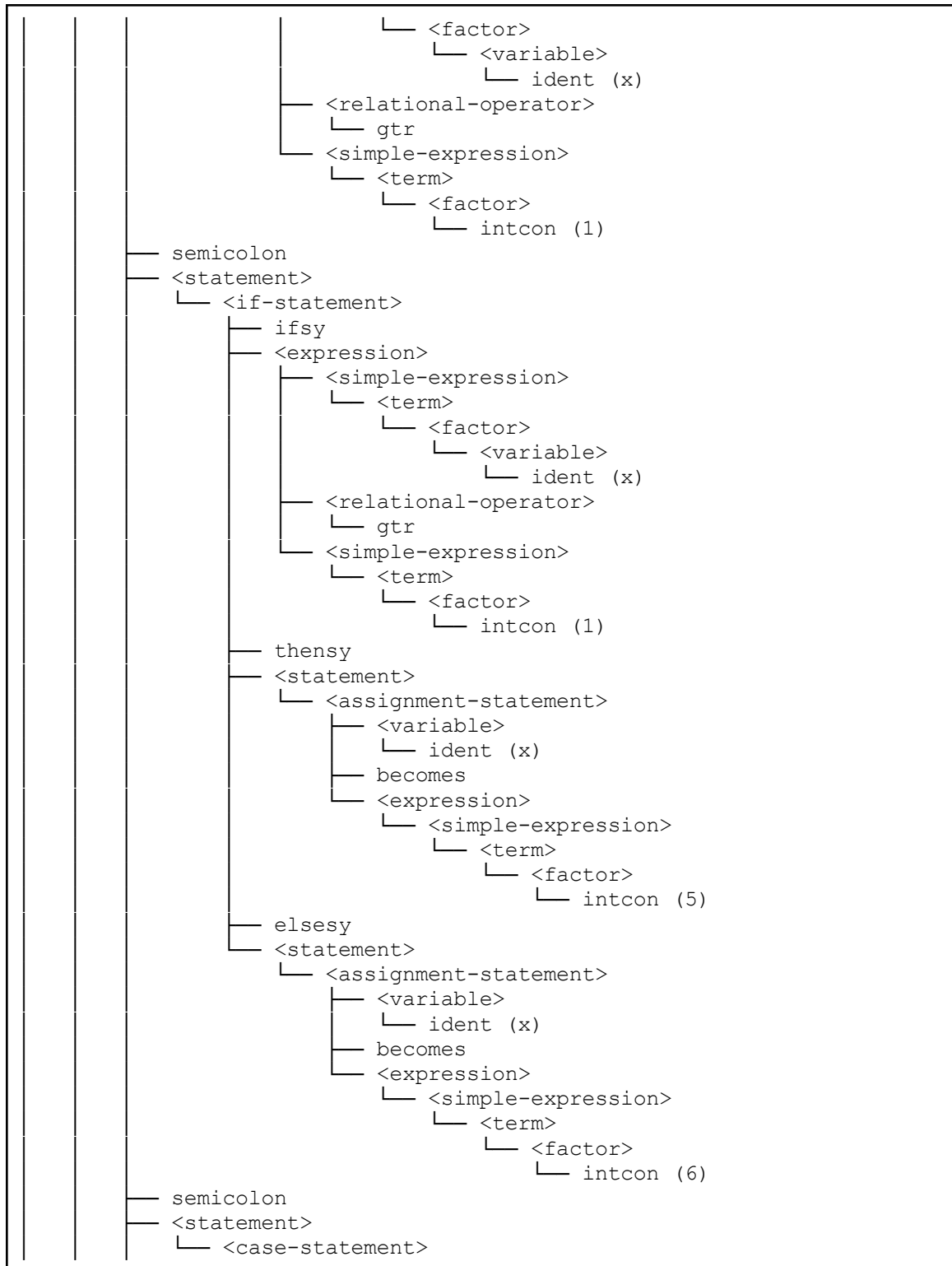
```

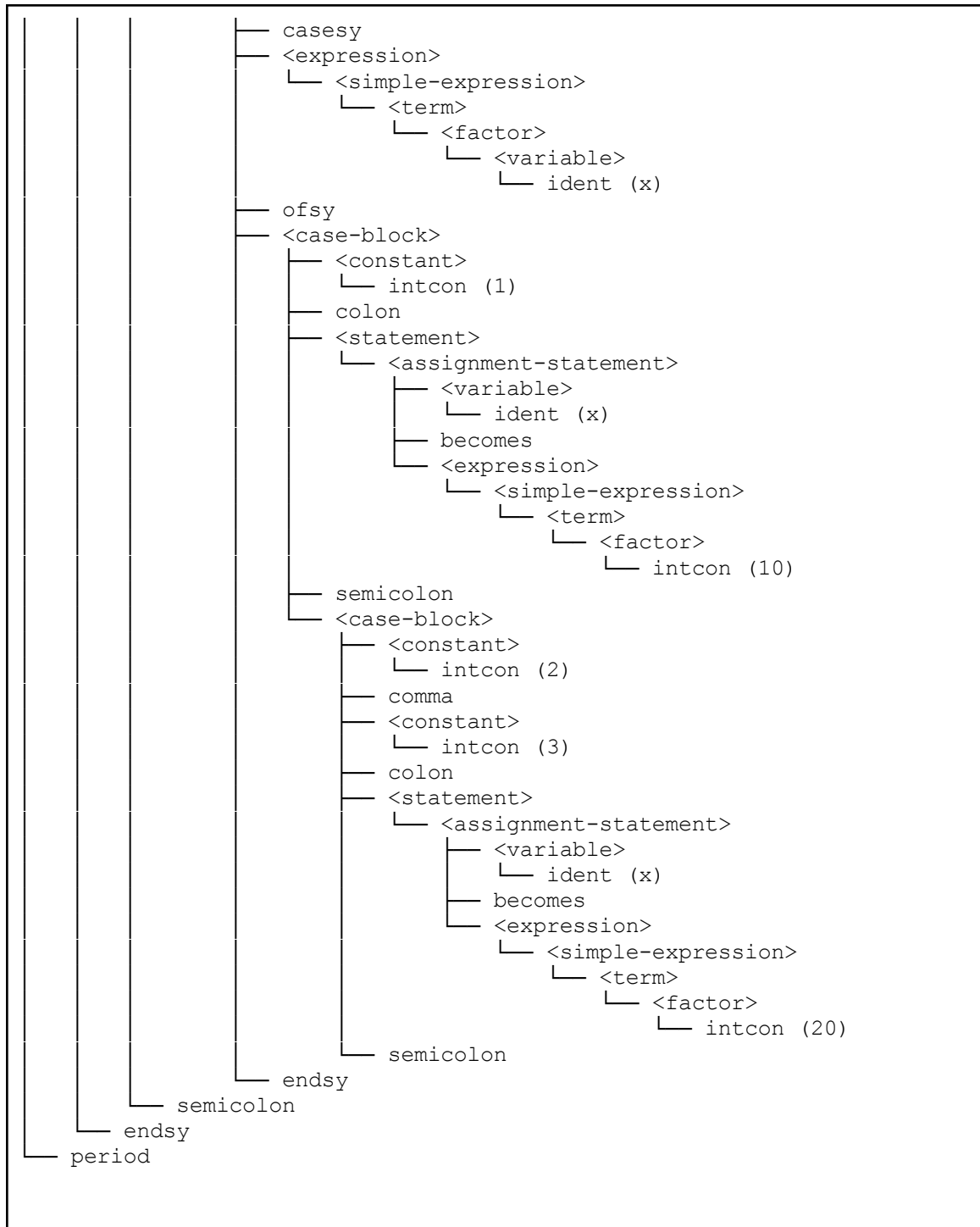
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (ControlFlow)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   ├── ident (i)
│       │   ├── comma
│       │   └── ident (x)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   └── <statement-list>
│       └── <statement>
│           └── <assignment-statement>
│               ├── <variable>
│               │   └── ident (x)

```









D. Pengujian 4

Input

```

program ArraysRecords;

var
  m: array[1..2] of array[1..2] of integer;
  p: record
    age: integer;
    name: string
  end;

begin
  m[1][2] := 7;
  p.age := m[1][2];
  p.name := 'abc';
end.

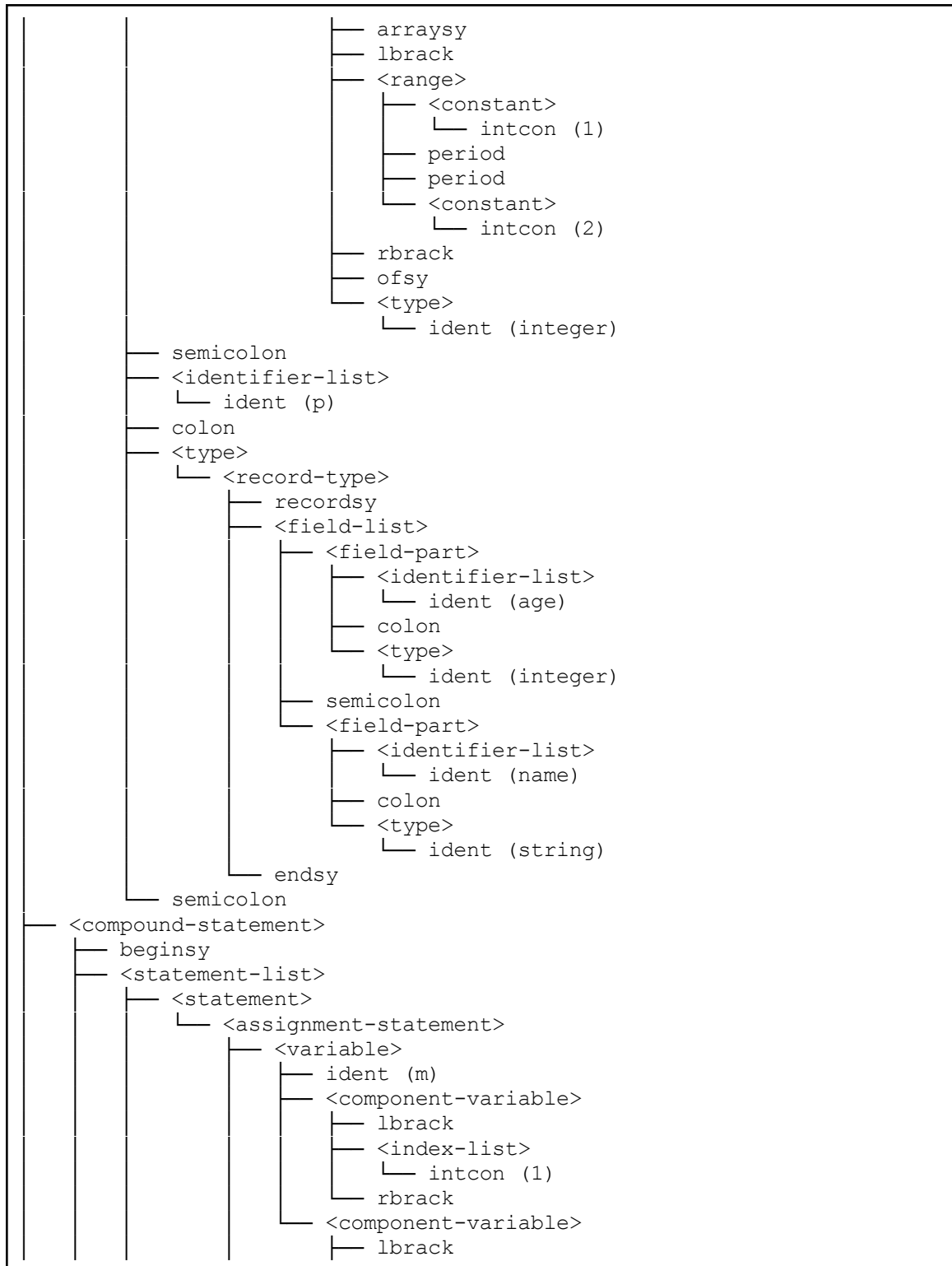
```

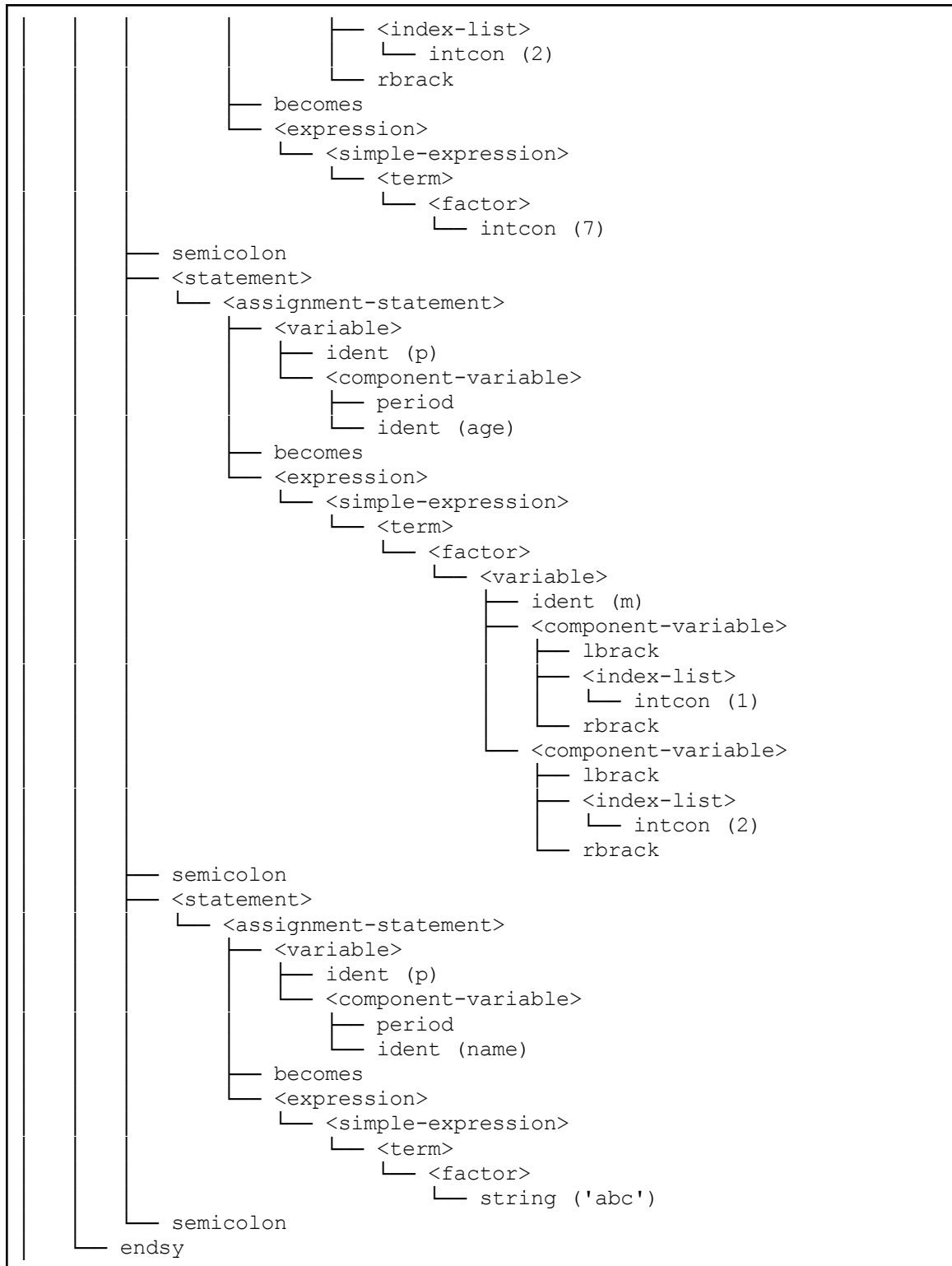
Output

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident (ArraysRecords)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident (m)
│       ├── colon
│       ├── <type>
│       │   └── <array-type>
│       │       ├── arraysy
│       │       ├── lbrack
│       │       ├── <range>
│       │       │   ├── <constant>
│       │       │   │   └── intcon (1)
│       │       │   ├── period
│       │       │   ├── period
│       │       │   └── <constant>
│       │       │       └── intcon (2)
│       │       ├── rbrack
│       │       ├── ofsy
│       │       └── <type>
│       │           └── <array-type>

```





└ period

E. Pengujian 5

Input

```
program Everything;

var
  i, j: integer;
  x: real;
  ch: char;
  e: (A, B, C);
  r: 1..3;
  mat: array[1..3] of array[1..3] of integer;
  rec: record
    age: integer;
    code: integer
  end;

procedure Update(a, b: integer; v: array[1..3] of integer);
begin
  v[1] := a + b;
  v[2] := (a * 2) div 2;
  v[3] := (v[1] + v[2]) mod 3;
end;

function NextVal(n: integer): integer;
begin
  NextVal := n + 1;
end;

begin
  i := 1;
  j := -i + 2 * 3 div 2 mod 2;
  x := 6 / 2;
  ch := 'z';
  mat[1,2] := j;
  rec.age := mat[1][2];
  rec.code := NextVal(rec.age);

  if not (j <= 0) or (i <> 0) then
    j := j + 1
  else
    j := j - 1;

  case j of
    1: j := 10;
```

```

    2, 3: j := 20;
end;

while (j > 0) and (i < 10) do
    j := j - 1;

repeat
    j := j + 1;
until j >= 2;

for i := 1 to 3 do
    Update(i, j, mat[1]);

for i := 3 downto 1 do
    j := j + i;

Update(i, j, mat[2]);
writeln('done', j, x, ch);
end.

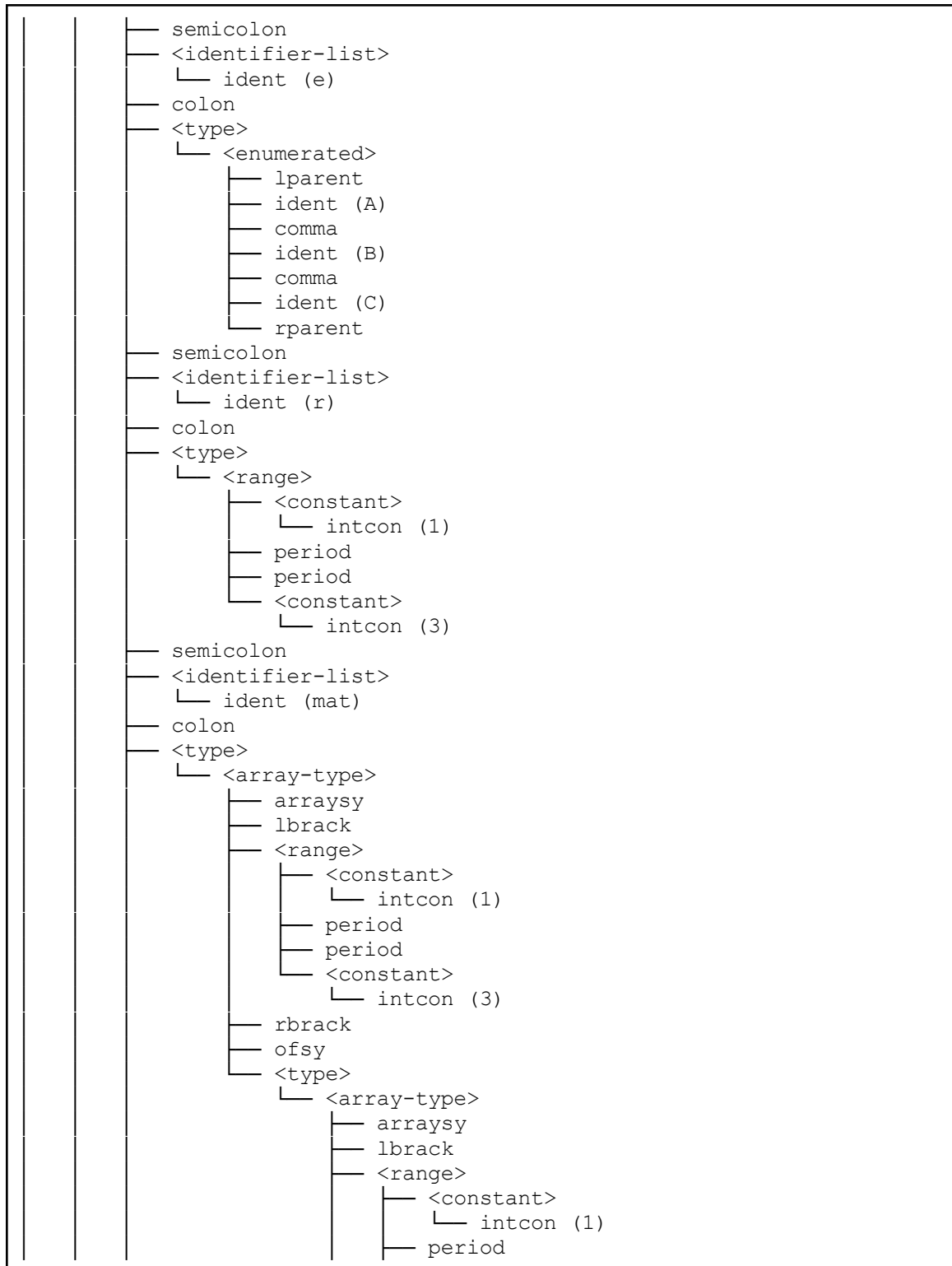
```

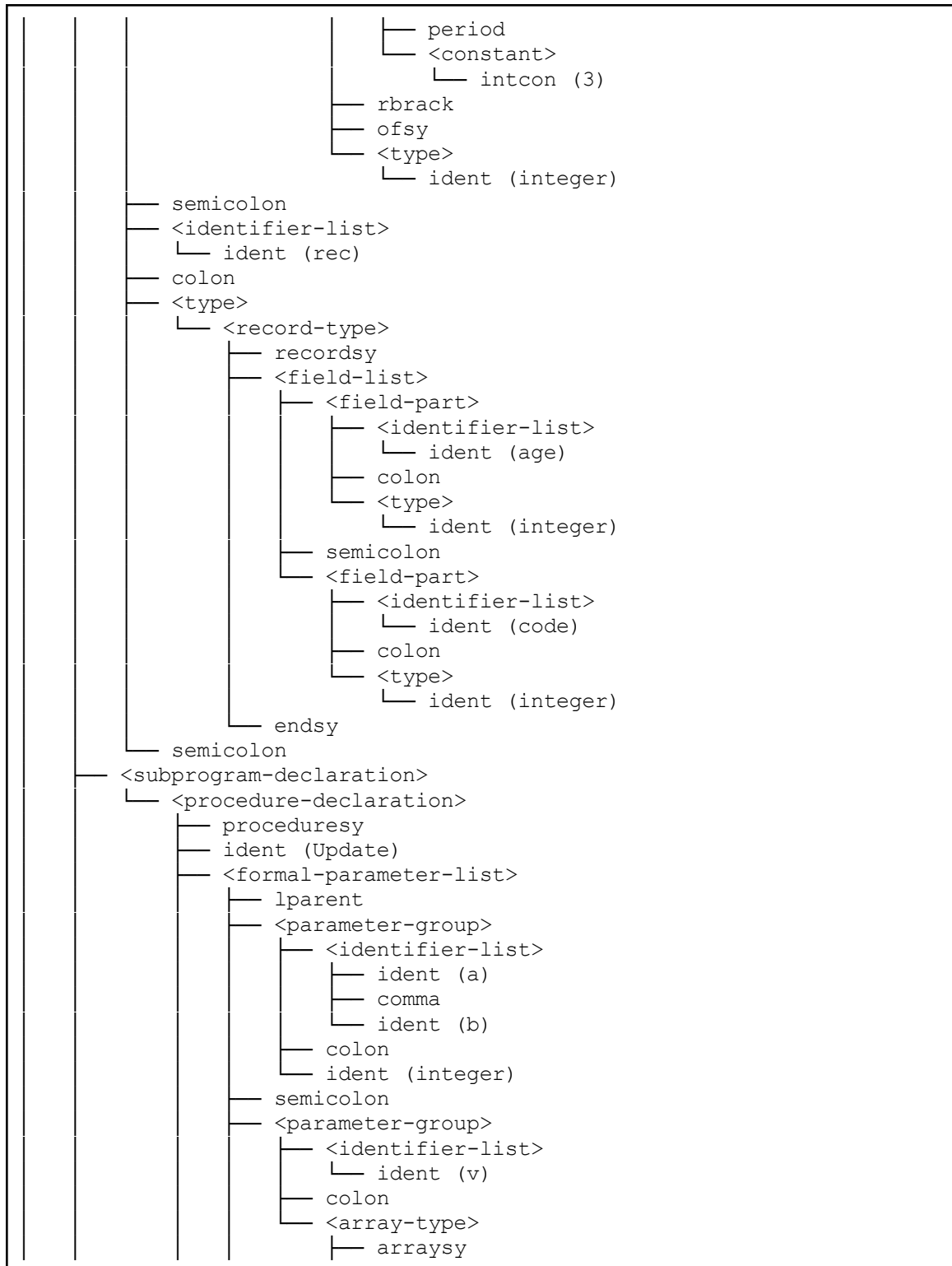
Output

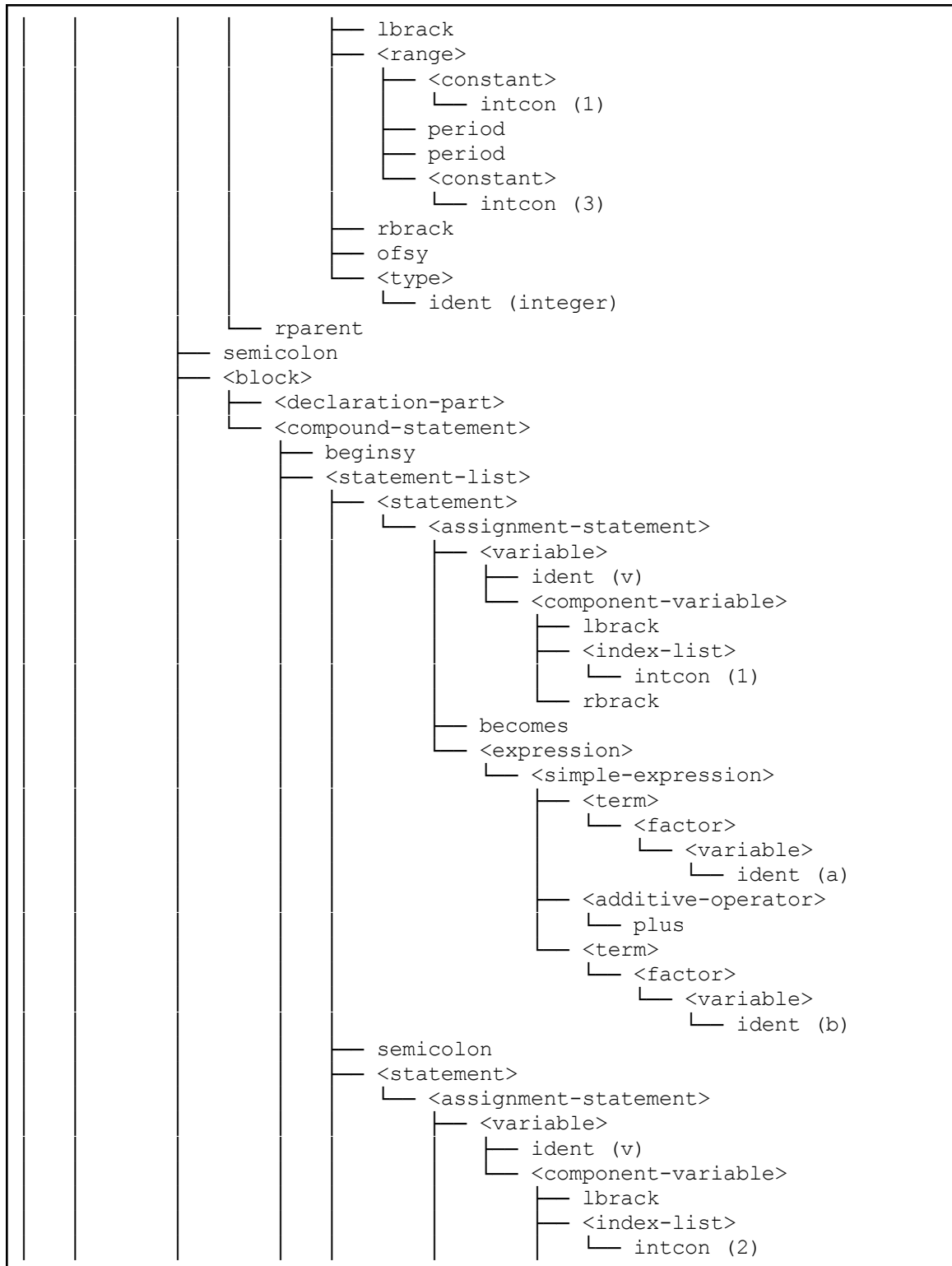
```

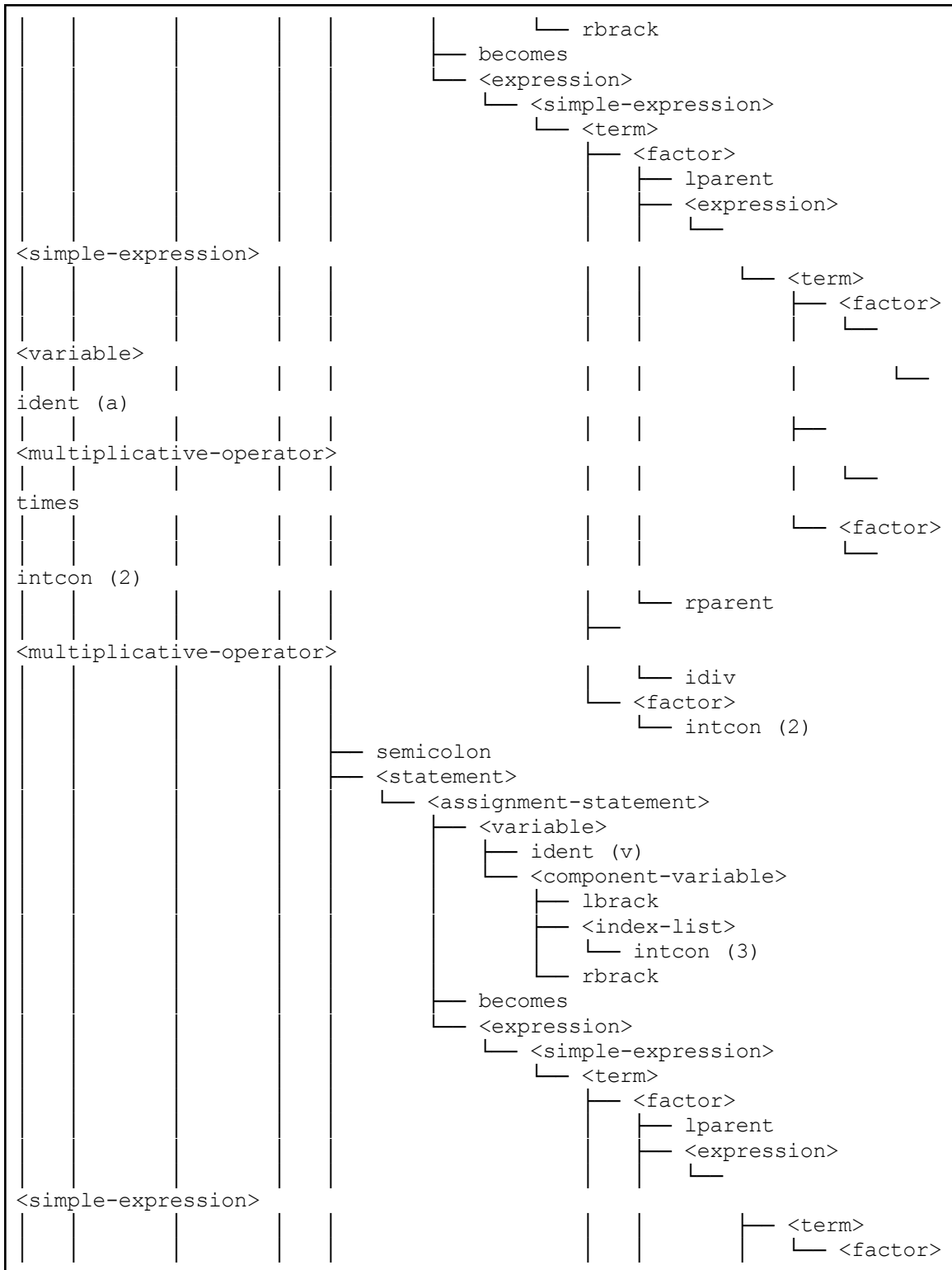
<program>
├── <program-header>
│   ├── programsy
│   ├── ident (Everything)
│   └── semicolon
├── <declaration-part>
│   ├── <var-declaration>
│   │   ├── varsy
│   │   ├── <identifier-list>
│   │   │   ├── ident (i)
│   │   │   ├── comma
│   │   │   └── ident (j)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (integer)
│   │   ├── semicolon
│   │   ├── <identifier-list>
│   │   │   └── ident (x)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (real)
│   │   ├── semicolon
│   │   ├── <identifier-list>
│   │   │   └── ident (ch)
│   │   ├── colon
│   │   ├── <type>
│   │   │   └── ident (char)

```

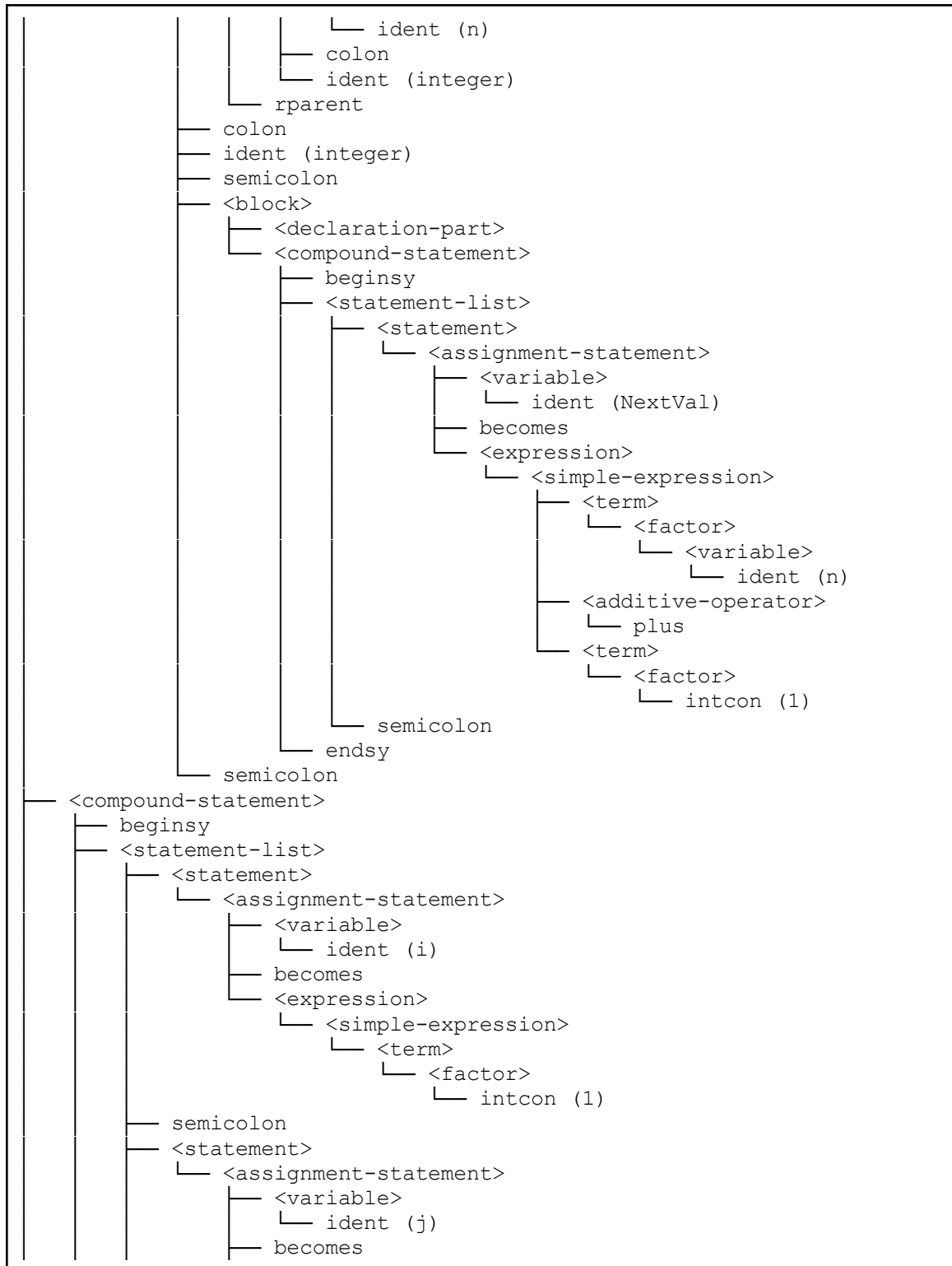


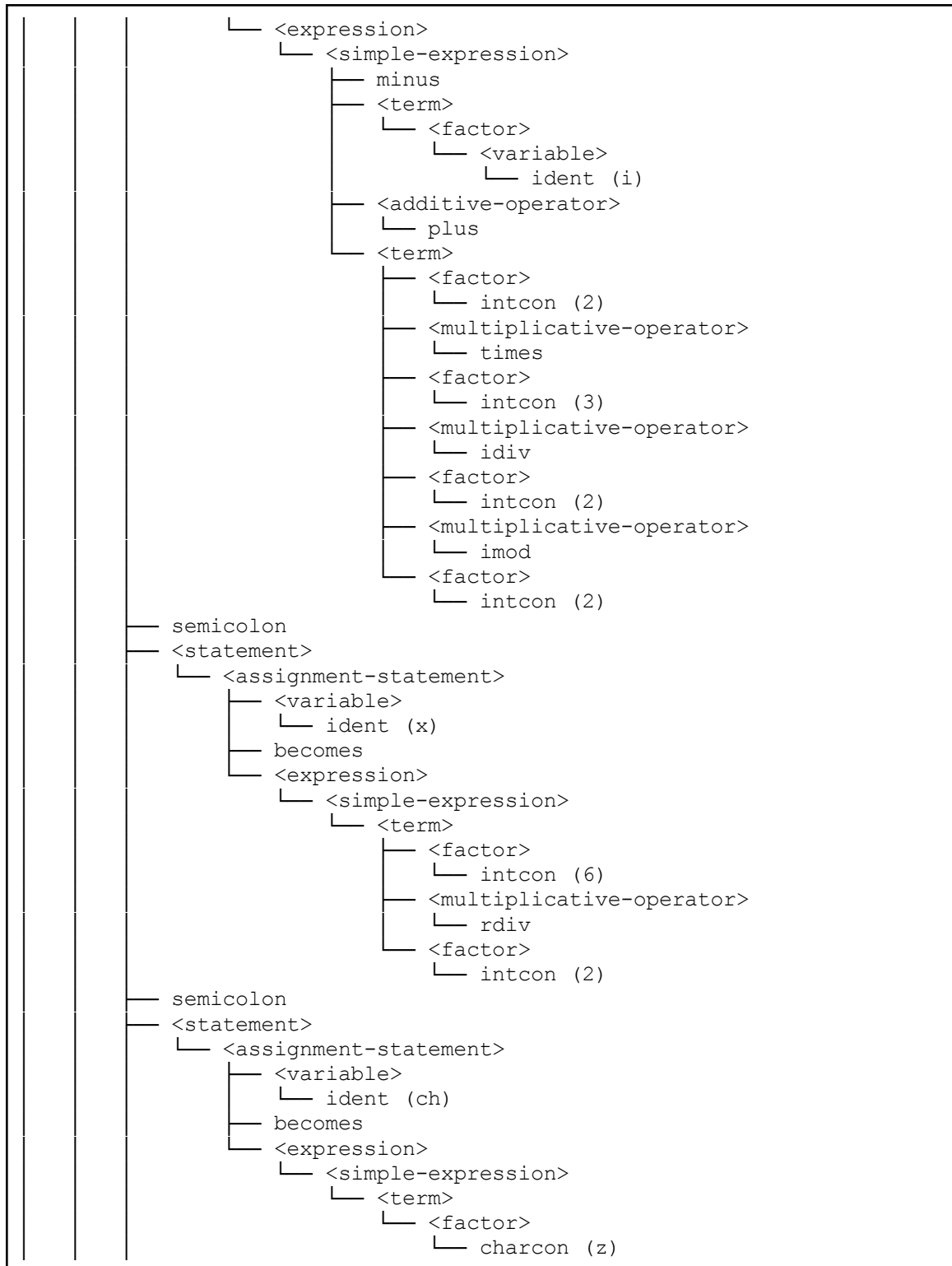


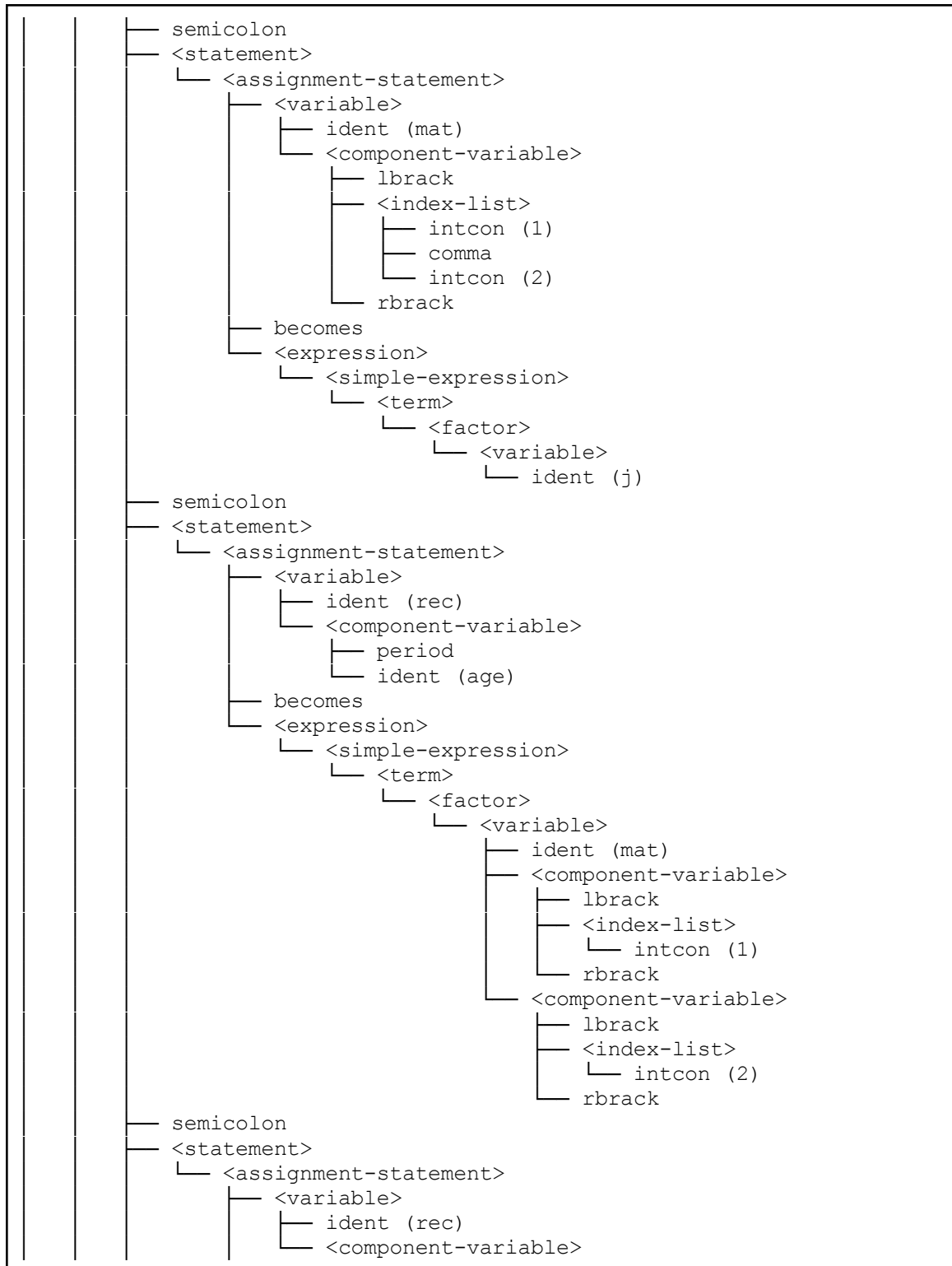


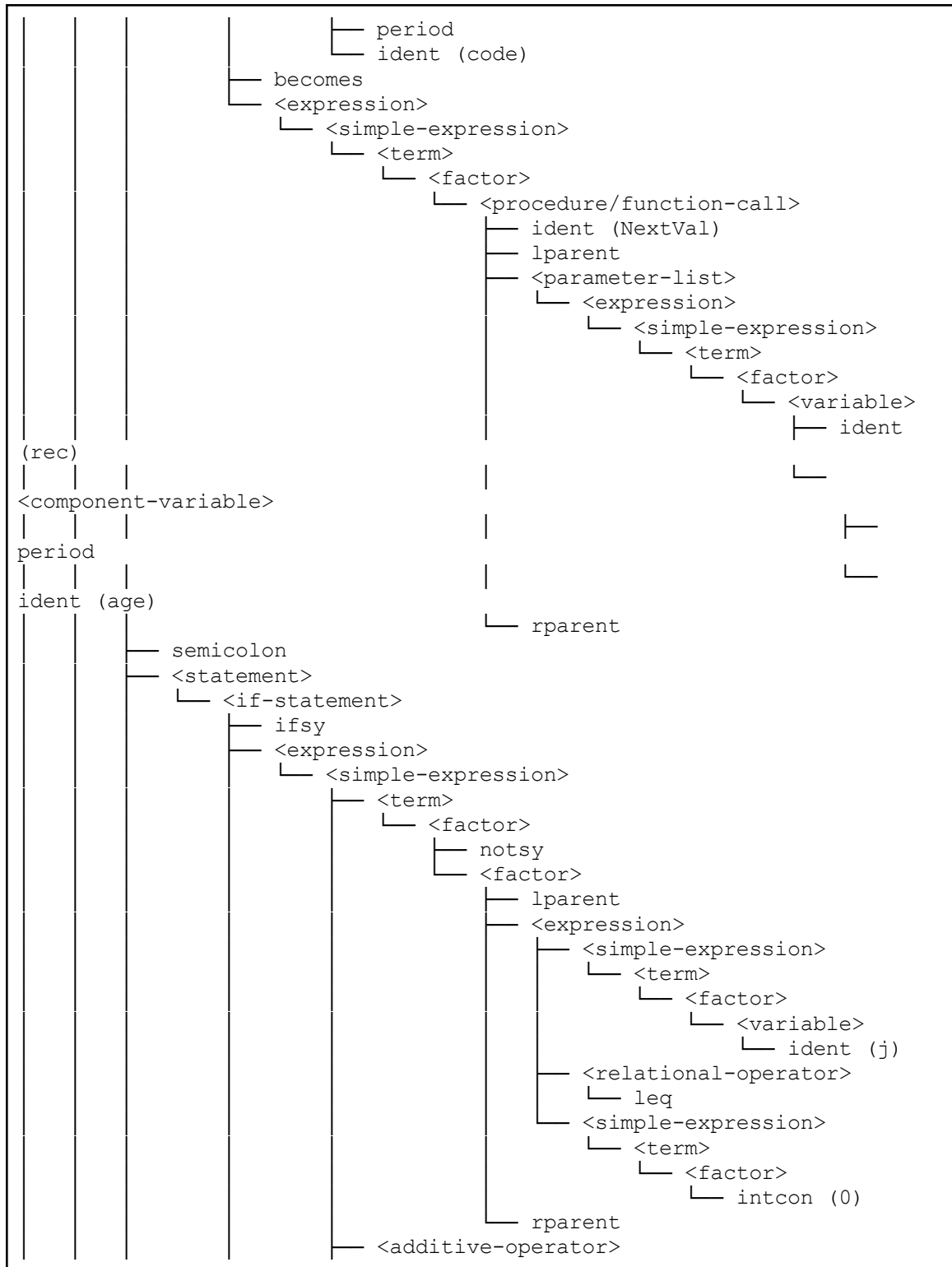


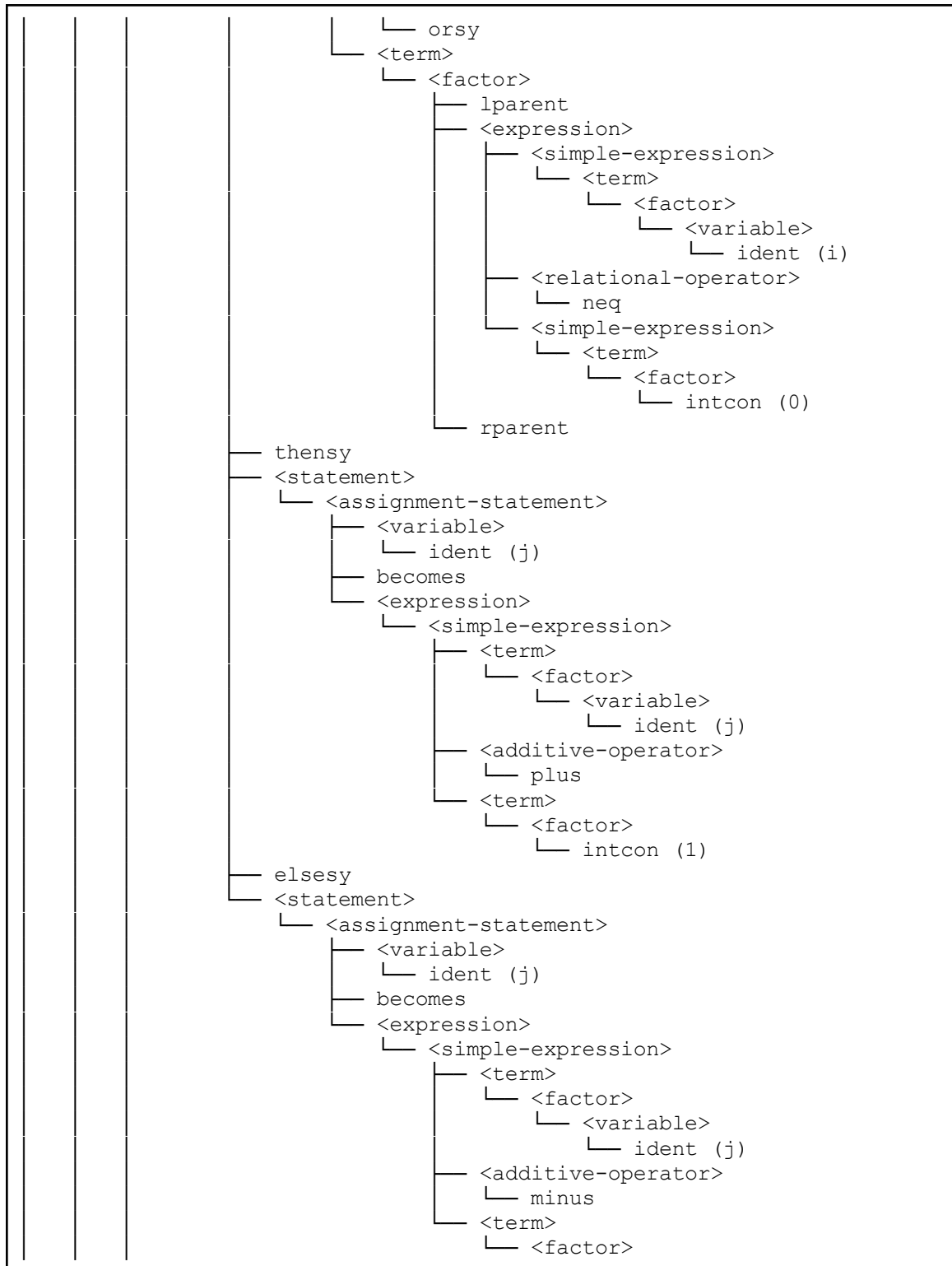


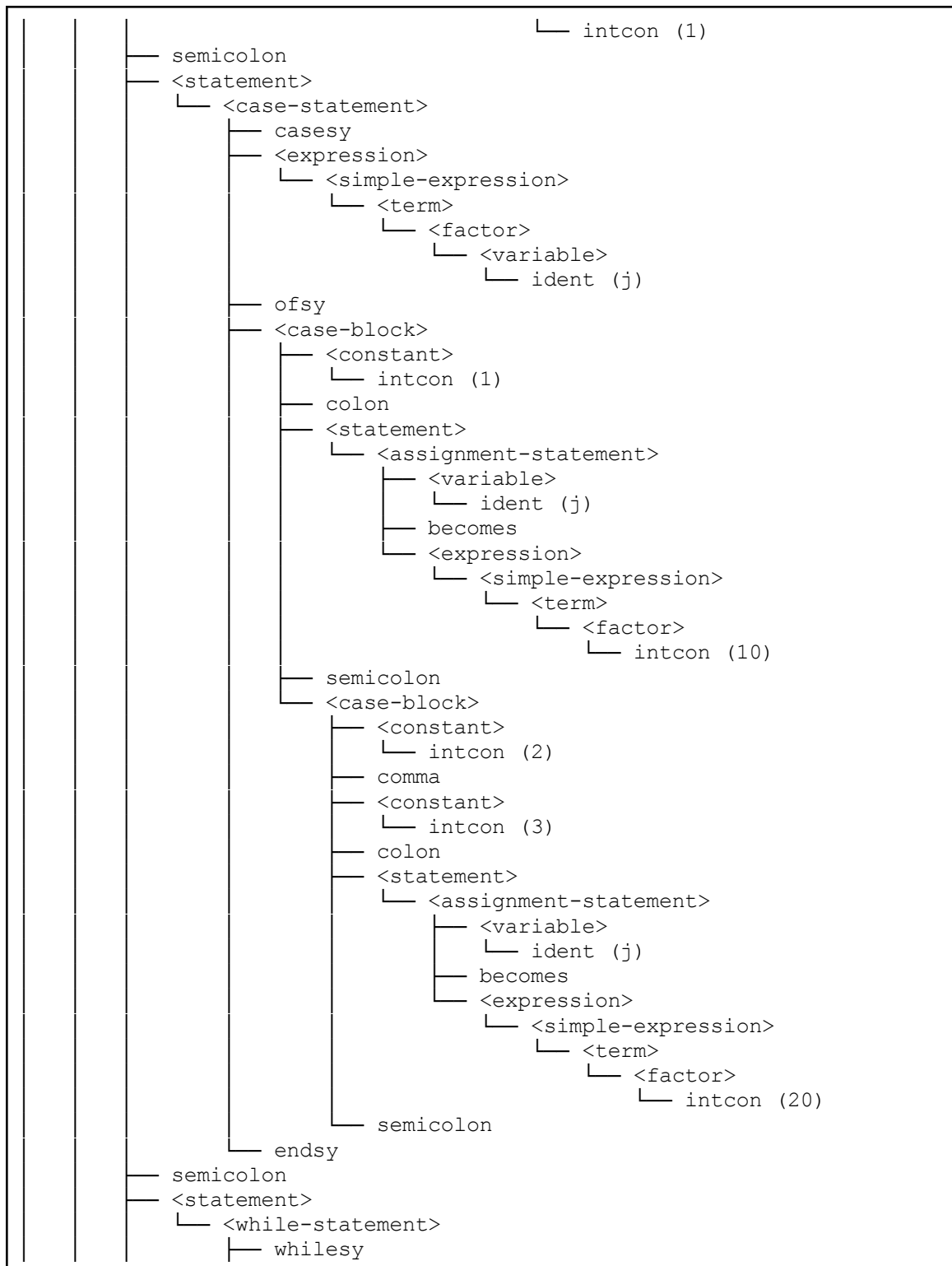


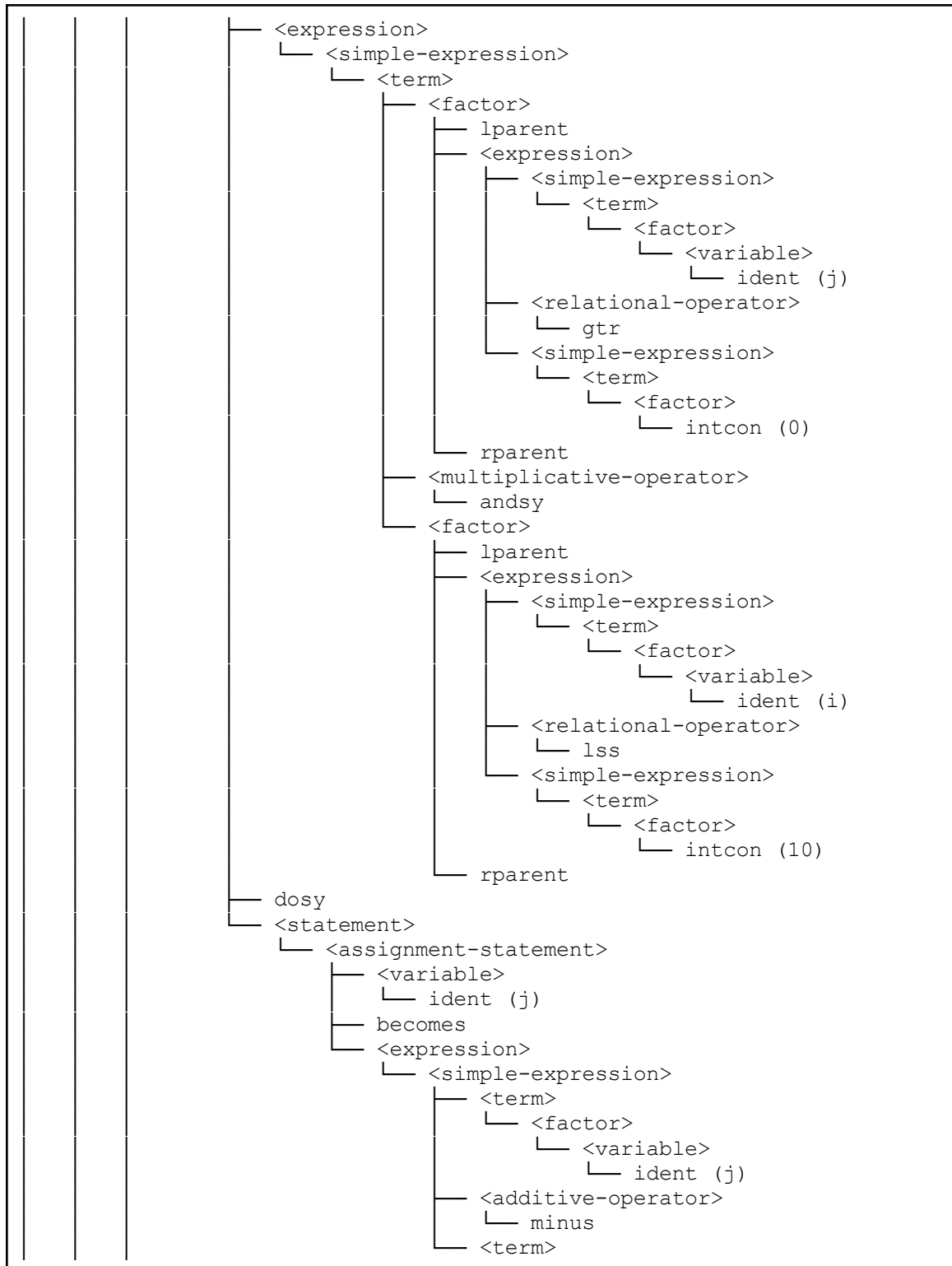


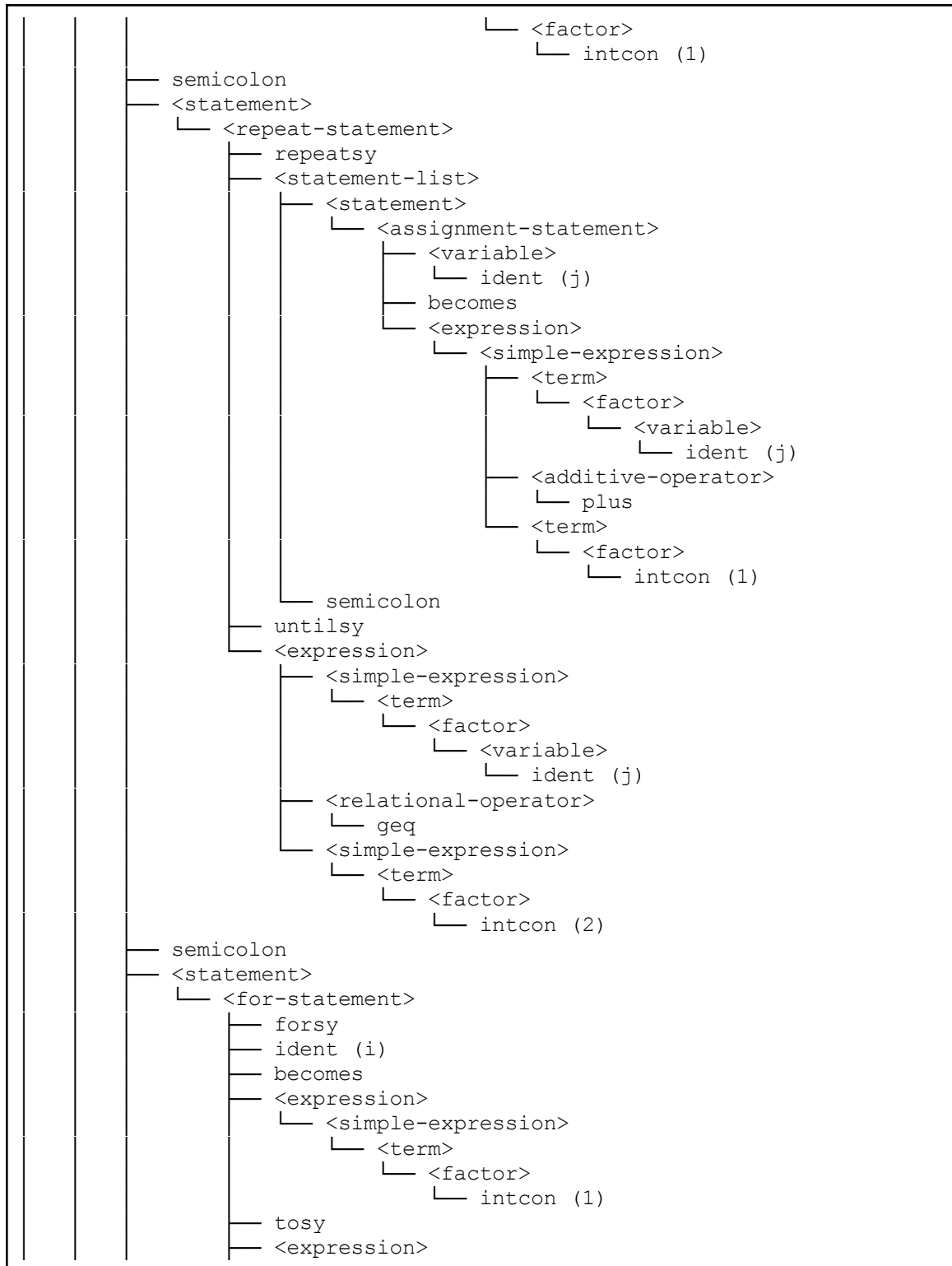


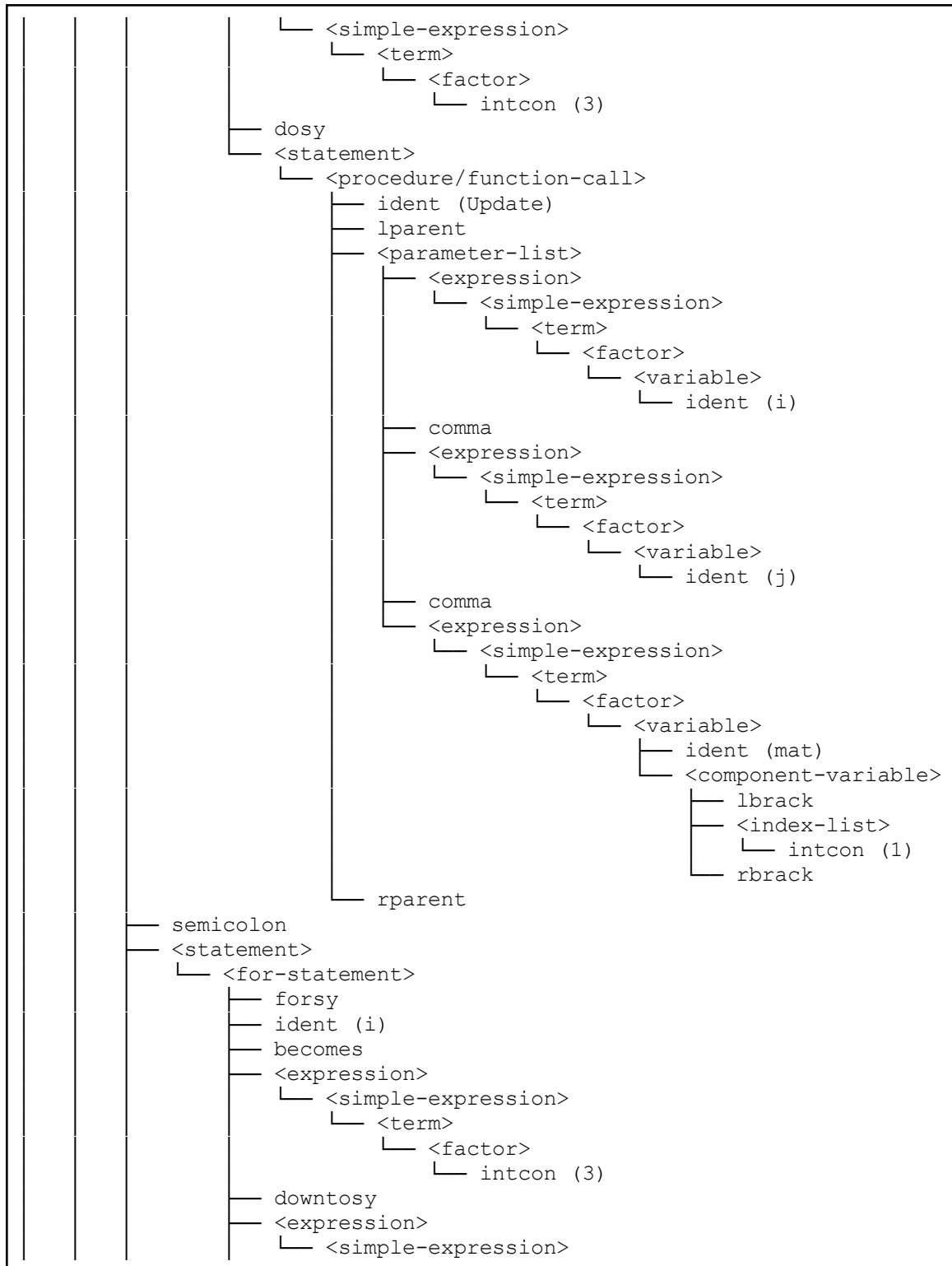


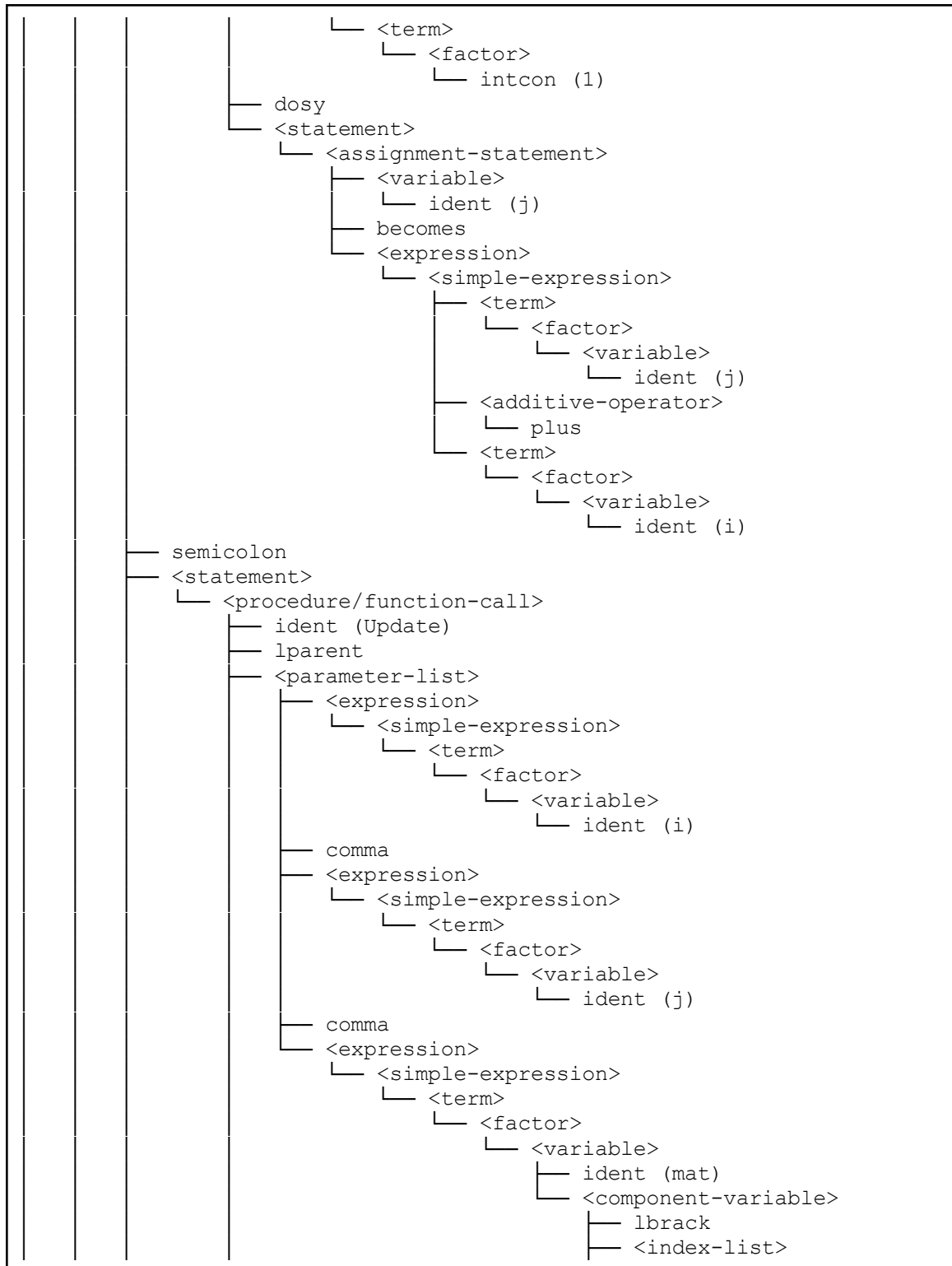


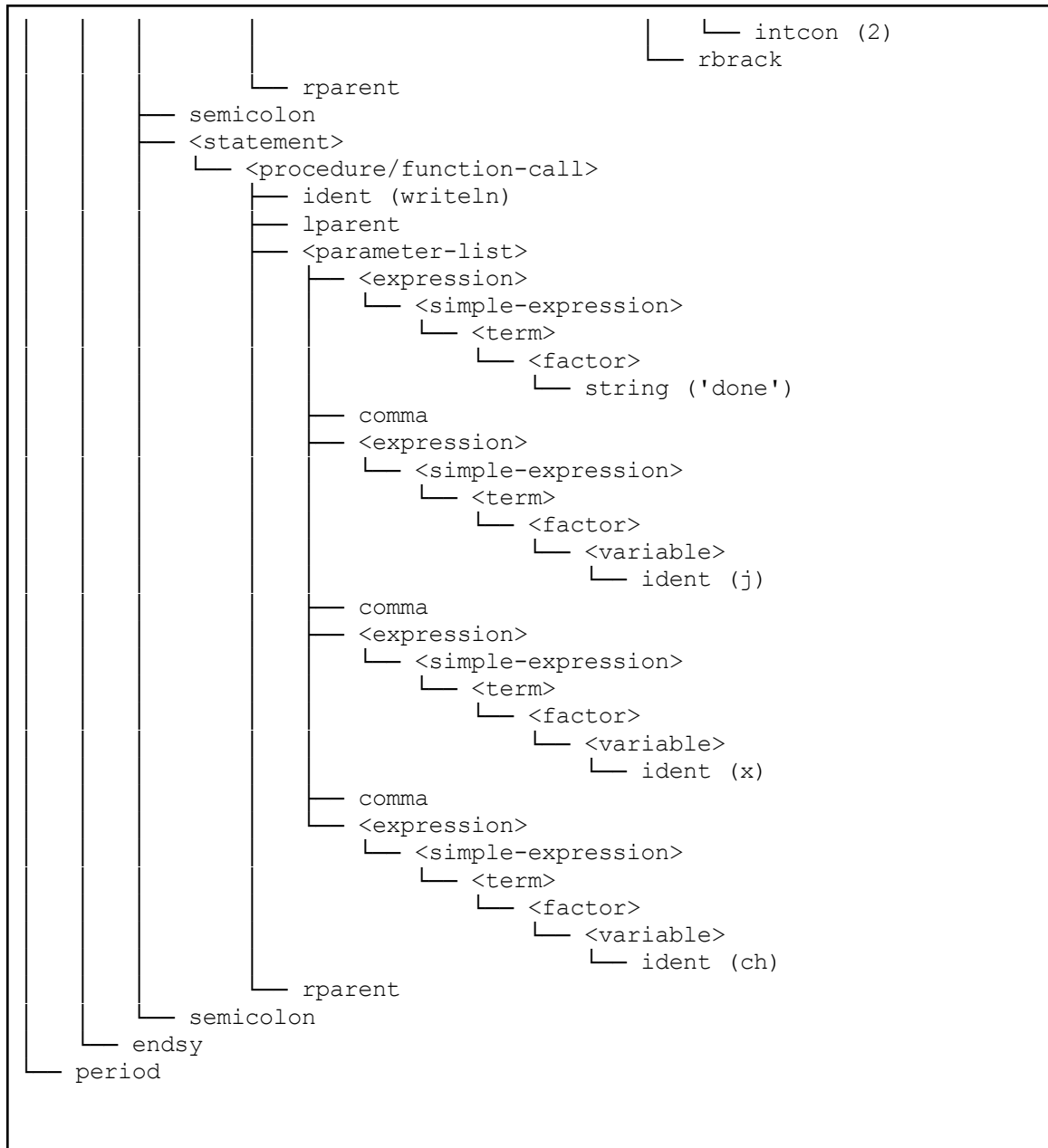












F. Pengujian 6

Input

```
program MissingSemicolon
```

```

var
  a: integer;

begin
  a := 1;
end.

```

Output

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident (MissingSemicolon)
│   └── [missing:semicolon]
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident (a)
│       ├── colon
│       ├── <type>
│       │   └── ident (integer)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   ├── <statement-list>
│   │   ├── <statement>
│   │   │   ├── <assignment-statement>
│   │   │   │   ├── <variable>
│   │   │   │   │   └── ident (a)
│   │   │   │   ├── becomes
│   │   │   │   └── <expression>
│   │   │   │       ├── <simple-expression>
│   │   │   │       │   └── <term>
│   │   │   │       │       └── <factor>
│   │   │   │       │           └── intcon (1)
│   │   └── semicolon
│   └── endsy
└── period

--- Syntax Errors ---
Syntax error: expected "semicolon", got "varsy" (near token line 4)

```

G. Pengujian 7

Input

```
program BadConst;  
  
const  
  A == ;  
  
begin  
end.
```

Output

```
<program>  
├── <program-header>  
│   ├── programsy  
│   ├── ident (BadConst)  
│   └── semicolon  
├── <declaration-part>  
│   └── <const-declaration>  
│       ├── constsy  
│       ├── ident (A)  
│       ├── eql  
│       ├── <constant>  
│       └── semicolon  
├── <compound-statement>  
│   ├── beginsy  
│   ├── <statement-list>  
│   │   └── <statement>  
│   └── endsy  
└── period  
  
--- Syntax Errors ---  
Syntax error: expected constant, got "unknown (==)"
```

H. Pengujian 8

Input

```
program BadType;  
  
var  
  a: array[1..3 of integer;
```

```

begin
  a[1] := 10;
end.

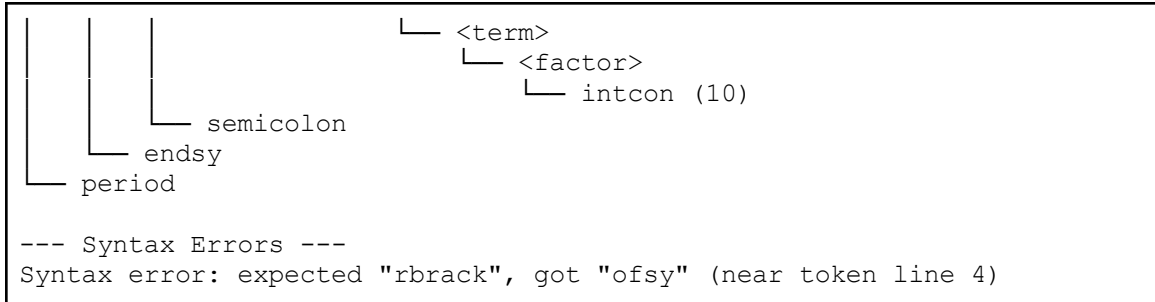
```

Output

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident (BadType)
│   └── semicolon
├── <declaration-part>
│   └── <var-declaration>
│       ├── varsy
│       ├── <identifier-list>
│       │   └── ident (a)
│       ├── colon
│       ├── <type>
│       │   └── <array-type>
│       │       ├── arraysy
│       │       ├── lbrack
│       │       ├── <range>
│       │       │   ├── <constant>
│       │       │   │   └── intcon (1)
│       │       │   ├── period
│       │       │   ├── period
│       │       │   └── <constant>
│       │       │       └── intcon (3)
│       │       ├── [missing:rbrack]
│       │       ├── ofsy
│       │       └── <type>
│       │           └── ident (integer)
│       └── semicolon
├── <compound-statement>
│   ├── beginsy
│   └── <statement-list>
│       └── <statement>
│           └── <assignment-statement>
│               ├── <variable>
│               │   ├── ident (a)
│               │   └── <component-variable>
│               │       ├── lbrack
│               │       ├── <index-list>
│               │       │   └── intcon (1)
│               │       └── rbrack
│               ├── becomes
│               ├── <expression>
│               └── <simple-expression>

```

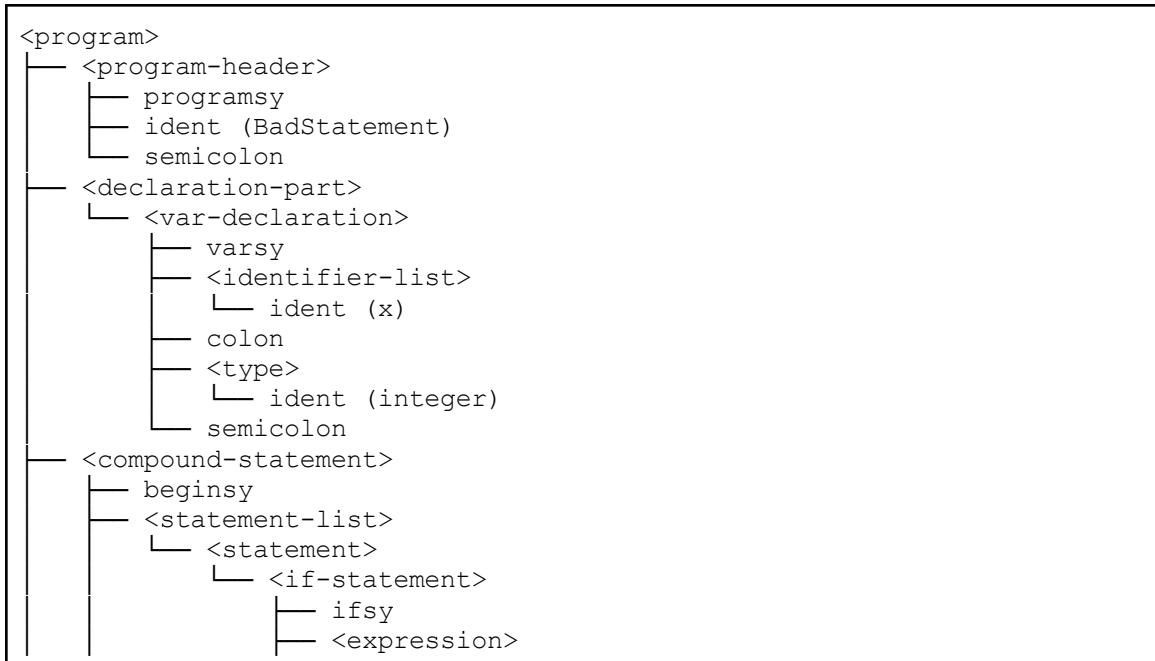


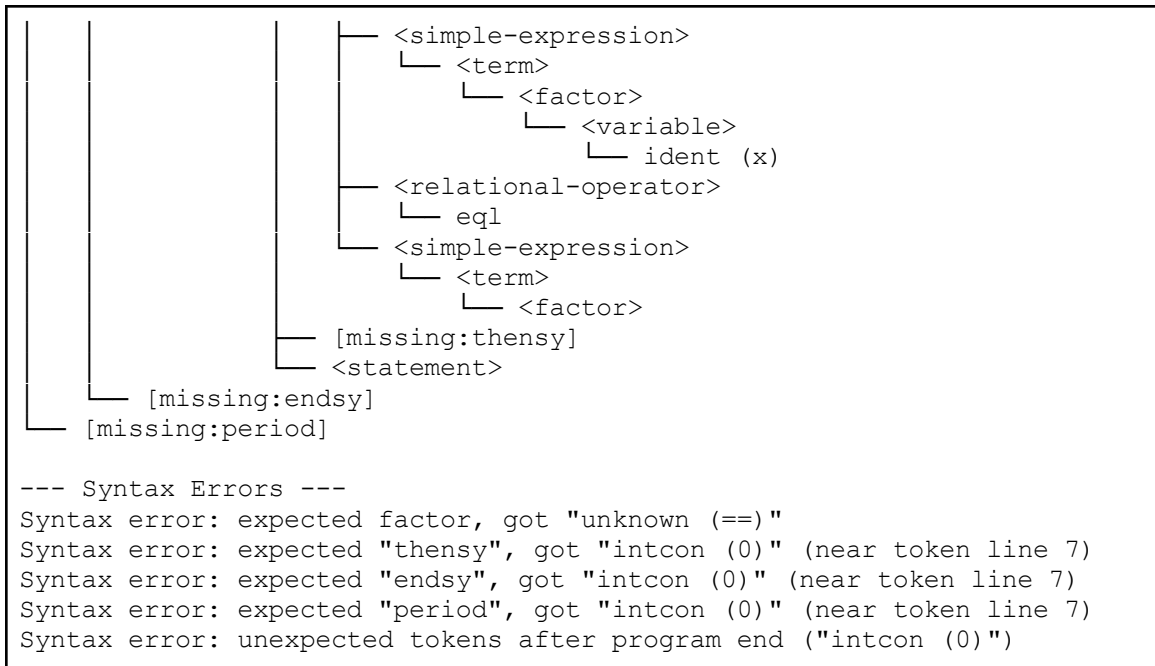
I. Pengujian 9

Input

```
program BadStatement;
var
  x: integer;
begin
  if x == 0 x := 1;
end.
```

Output





J. Pengujian 10

Input

```

program BadEnd;

begin
  writeln('hello');
end

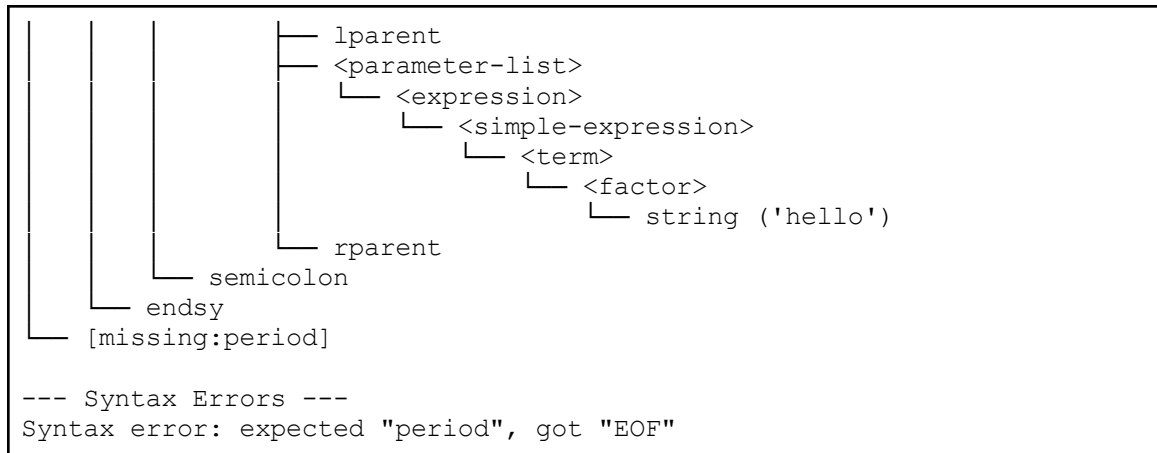
```

Output

```

<program>
├── <program-header>
│   ├── programsy
│   ├── ident (BadEnd)
│   └── semicolon
├── <declaration-part>
├── <compound-statement>
│   ├── beginsy
│   └── <statement-list>
│       ├── <statement>
│       │   └── <procedure/function-call>
│       │       └── ident (writeln)

```

IV. Kesimpulan dan Saran

A. Kesimpulan

Implementasi program parser untuk bahasa Arion yang kelompok kami buat telah berhasil memenuhi tahapan Syntax Analysis. Program yang dirancang mampu menginterpretasi token-token dari Lexer menjadi struktur pohon logis (Parse Tree) melalui algoritma top-down berbasis Recursive Descent. Sesuai dengan saran pada milestone sebelumnya, kami telah merevisi Lexer untuk menyimpan token ke dalam struktur data dynamic array (`std::vector`) pada saat runtime untuk memudahkan integrasi.

Hasil pengujian juga menunjukkan bahwa keseluruhan hierarki tata bahasa telah berhasil dipetakan, termasuk penanganan tingkat prioritas operator matematika yang diproses berjenjang. Selain itu, kelancaran proses parsing didukung dengan error handling yang tidak langsung menghentikan program, melainkan menyisipkan node `[missing:token]` saat menemukan kesalahan sintaksis.

B. Saran

Untuk pengembangan pada *milestone* berikutnya, kami berencana untuk menyempurnakan struktur pohon yang dihasilkan. *Parse Tree* saat ini masih menyimpan seluruh detail sintaksis (termasuk token seperti titik koma atau kata kunci struktural). Oleh karena itu, sangat disarankan untuk melakukan transformasi *Parse Tree* ini menjadi *Abstract Syntax Tree* (AST) untuk efisiensi di tahapan kompilasi selanjutnya.

Selain itu, dari sisi implementasi *error recovery*, meskipun error handling saat ini sudah bekerja dengan baik, kami berencana memperluas mekanisme "*panic-mode*" di dalam parser sehingga program bisa mengabaikan blok kode yang bermasalah dan kembali beroperasi normal dari titik sinkronisasi tertentu. Ini akan menghindari kesalahan beruntun yang

dihasilkan kesalahan tunggal pada kasus kompleks Penggunaan manajemen memori yang lebih aman juga patut dipertimbangkan untuk menjamin stabilitas *compiler* saat memproses kode berskala besar.

Dari sisi pemrosesan, algoritma *Recursive Descent* murni juga memiliki kelemahan, seperti tidak secepat algoritma lain dan kesulitan dalam menghadapi *grammar* yang bersifat *left-recursive*. Oleh karena itu, pemilihan algoritma *parsing* alternatif patut dipertimbangkan ke depannya. Pendekatan *bottom-up* (seperti *LR Parser*) atau pemanfaatan *parser generator* dapat dieksplorasi apabila tata bahasa Arion berkembang menjadi jauh lebih kompleks.

Referensi

"Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees"

<https://ruslanspivak.com/lrbasi-part7>

"Contoh Gambar Parse Trees"

https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

"Introduction to Automata Theory, Languages, and Computation 3rd Edition"

https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf

"Parsers"

<https://www.ibm.com/docs/en/integration-bus/10.0.0?topic=overview-parsers>

"Apa Itu Parser? Fungsi dan Perannya dalam Bahasa Pemrograman"

<https://idcloudhost.com/blog/apa-itu-parser/>

"Parsing - Introduction to Parsers"

<https://www.geeksforgeeks.org/compiler-design/introduction-of-parsing-ambiguity-and-parsers-set-1/>

"Parse Tree in Compiler Design"

<https://www.geeksforgeeks.org/compiler-design/parse-tree-in-compiler-design/>

"Parse Tree in Automata Theory"

https://www.tutorialspoint.com/automata_theory/automata_theory_parse_tree.htm

Lampiran

Github : [BST-Tubes-IF2224-2026](https://github.com/BST-Tubes-IF2224-2026)

Pembagian Tugas :

NIM	Persentase
13524019	25%
13524033	25%
13524043	25%
13524101	25%